

FSLog: A Federated Split Learning Framework for Privacy-Protected and Computation-Efficient Syslog Fault Diagnosis in IoT Networks

Ting Xu¹, Member, IEEE, Shenglin Zhang², Member, IEEE, Zeyu Che, Yongqian Sun³, Member, IEEE, Wenwei Gu⁴, Yicheng Sui, Sibao Xia, Zijong Que, Zhimin Chen, Weiwei Shang⁵, Senior Member, IEEE, Kundi Yao⁶, Jiayi Ma, and Dan Pei⁷, Senior Member, IEEE

Abstract—Logs are critical for ensuring operational reliability in Internet of Things (IoT) networks. With the widespread proliferation of IoT device networks, traditional log analysis becomes computationally intensive and exposes raw system logs (syslogs) to privacy risks, complicating fault diagnosis in cloud-integrated environments. To overcome these limitations, we propose *FSLog*, a novel federated split learning (FSL) framework aimed at IoT. It enables collaborative training while ensuring data privacy and alleviating the computational load on the client-side. First, a multilayer privacy protection mechanism combining feature filtering and irreversible intermediate activation protection mitigates reconstruction attacks. Second, a model-splitting strategy combining split learning (SL) and bidirectional encoder representations from transformers (Bert), termed SL-Bert, offloads the Transformer backbone to the vendor server (VS), while the client server (CS) retains lightweight embeddings and personalized heads, thereby reducing the local computational load. Third, to combat the adverse effects of nonindependent and identically distributed (non-IID) data on global model performance, we integrate component-driven log grouping with personalized fine-tuning, further enhanced by a synergistic bidirectional federated learning (FL)-exponential moving average (EMA) loop combining forward FedAvg and backward EMA. This FL-EMA synergy improves model generalization and adaptability, enabling efficient, privacy-preserving fault diagnosis in IoT networks. We conduct extensive experiments on a Dockerized cluster with two real datasets, achieving macro-averaged F1-scores (M-F1) of 0.95 and 0.96 for fault diagnosis, representing improvements of 10.6 and 6.0 percentage points over the baseline, respectively, while reducing client-side computational load (in FLOPs) by over 99.8%. Furthermore, the containerized deployment in IoT microservices demonstrates that it enables rapid root-cause analysis (RCA), effectively protecting enterprise data.

Index Terms—Bidirectional loop, computation offloading, fault diagnosis, federated split learning, privacy protection.

Received 20 December 2025; revised 3 May 2026 and 24 June 2026; accepted 25 July 2026. This work was supported by the National Natural Science Foundation of China under Grant 62272249 and Grant 62302244. (Corresponding author: Shenglin Zhang.)

Ting Xu, Shenglin Zhang, Zeyu Che, Yongqian Sun, Wenwei Gu, Yicheng Sui, Sibao Xia, Zijong Que, and Zhimin Chen are with Nankai University, Tianjin 300350, China (e-mail: zhangsl@nankai.edu.cn).

Weiwei Shang and Jiayi Ma are with the University of Waterloo, Waterloo, ON N2L 3G1, Canada.

Kundi Yao is with the Faculty of Engineering and Applied Science, Ontario Tech University, Oshawa, ON L1G 0C5, Canada.

Dan Pei is with the Department of Computer Science and Technology, Tsinghua University, Beijing 100084, China.

Digital Object Identifier 10.1109/JIOT.2026.3720046

NOMENCLATURE

CEs	Customer enterprises.
DV	Device vendor.
VS	Vendor server.
CS	Client Server.

I. INTRODUCTION

WITH the rapid advancement of Internet of Things (IoT)-served cloud computing and big data technologies, the technical complexity of IoT device networks has intensified significantly. The increasing complexity means that any minor fault [1] in IoT can lead to service interruptions or degradation for small- and medium-sized CEs (e.g., telecommunications and e-commerce). Consequently, maintenance as a service (MaaS) [2] has recently received significant attention from the CE. The abbreviations used in this article are listed in the Nomenclature. The DV uses a professional and efficient VS that can provide uninterrupted 24/7 services to acquire predictive maintenance tasks in this cloud-integrated IoT network. More precisely, the DV provides round-the-clock system management, along with real-time recording of system logs (syslogs [3]) for operations and maintenance (O&M) staff to diagnose faults for CEs. In this article, fault diagnosis includes fault detection and root-cause analysis (RCA).

In IoT-cloud networks [4], effectively managing syslogs generated by IoT nodes is critical. Inherently, these syslogs exhibit nonindependent and identically distributed (non-IID) characteristics [5] driven by both interclient and intraclient heterogeneity. Specifically, interclient heterogeneity [6] arises from varying device models, hardware configurations, and operational scenarios across different enterprises. Meanwhile, intraclient heterogeneity [7] manifests even within the same customer site, where syslogs produced by different underlying components (e.g., CPU, memory, and network port) present distinct semantic patterns. A key issue occurs if many end nodes fail simultaneously in a short time (e.g., $n_{11}, \dots, n_{1m}, \dots, n_{\Omega 1}, \dots, n_{\Omega q}$), where Ω denotes the identifier for different CSs, and m and q represent the respective node counts corresponding to their CS. The client-side cannot identify the fault-related syslog keyword (see Fig. 1, marked in bold and red font) in a timely manner. To efficiently locate

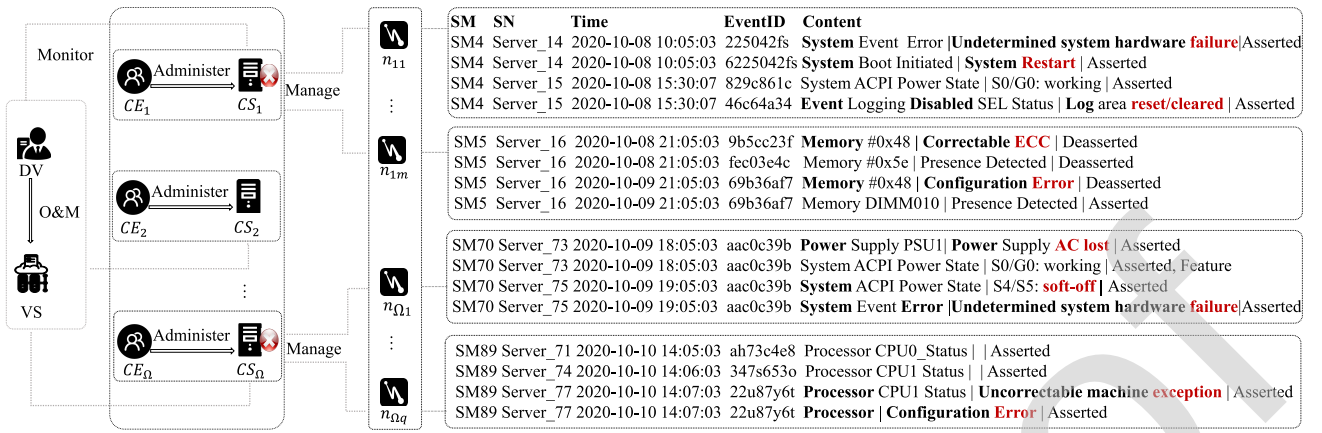


Fig. 1. Diagnosing fault cases based on syslog in the three-tier architecture of VS-CS node in MaaS.

and diagnose fault cases, the DV operates a VS that centrally manages many customer agent nodes and provides O&M capabilities. Meanwhile, the CE delegates the management of several server clusters (e.g., $CS_1, CS_2, \dots, CS_\Omega$) to the DV, while each cluster retains local control. That is, the CE manages local end nodes via agent nodes and uses their CS to upload syslogs to the VS for centralized analysis. IoT devices must communicate with an aggregator VS to perform collaborative training. This interaction between VS and CS can be effectively analyzed with federated learning (FL) [8], [9], [10], which enables the O&M staff to accurately diagnose faults.

Previous works [11], [12], [13], [14] apply FL to fault detection. Typically, FL [15] is used to train a global model in collaboration with each CE. However, given the excessive data generation by IoT devices [16] and the fact that IoT devices are constrained in terms of memory, computing power, and storage bandwidth, these methods raise corresponding concerns regarding privacy [17], security [18], [19], [20], and computational resources [4]. First, the frequent communication between CS and VS will lead to exhaustion of local computing resources, as many CE lack the processing capacity required to support vendor-led model training [21]. Second, commercial CE typically refuse to share raw logs with the DV, so the global model [22] cannot access high-quality training data. Even if some companies allow data sharing, data transmission poses security risks. Thus, FL alone is inadequate for MaaS-based IoT fault diagnosis, as it prioritizes global optimization while leaving client computation and raw-log privacy unresolved.

To address these issues, existing split learning (SL) methods [23], [24], [25] are widely used to build privacy-preserving distributed systems. These methods partition the model into client-side (e.g., CS) and server-side (e.g., VS) submodels. The client-side transmits intermediate activations (feature representations) [26] rather than full parameters [27] or raw data [11], thereby reducing leakage risk (e.g., Peek SplitNN [28]). Moreover, SL can offload compute-intensive tasks [29] to the server-side, alleviating client-side resource pressure. Compared with FL, SL provides model partitioning and computation offloading, but it does not inherently provide cross-client aggregation or personalized adaptation for heterogeneous CE environments.

Although IoT interconnects physical devices for automation, remote monitoring, and real-time response, it also introduces serious data-breach risks. The potential of federated split learning (FSL) [30], which integrates federated learning with split learning, to achieve fault diagnosis. For instance, SplitFed [24] combines SL with federated aggregation for parallel training across multiple clients, while FedBert [25] extends this paradigm to pretraining tasks by splitting the bidirectional encoder representations from transformers (Bert) model between local devices and a central server. However, these FSL frameworks mainly serve as general-purpose learning protocols and do not incorporate domain-specific components for syslog fault diagnosis. Although FSL collaborative training is promising, three key challenges remain in practical IoT deployments.

- 1) *Attack-Resistant Intermediate Representation*: Even though intermediate activations may be exchanged between the CS and VS, syslogs contain sensitive information, making them vulnerable to privacy extraction through inference or reconstruction attacks.
- 2) *CSs With Limited Compute Resources*: Although SL alleviates computational pressure [31], [32], [33] on the client-side, its deployment on resource-constrained devices remains limited by split-point selection. This is due to the increased computational burden imposed by a later split point, which strains the capabilities of weaker IoT devices.
- 3) *Client Drift*: IoT devices exhibit non-IID data, which induces client drift that degrades the performance of the aggregated global model.

Motivated by these challenges, we focus on lightweight algorithm design for resource-constrained IoT devices and the fusion with deep learning to achieve higher fault diagnosis accuracy. We propose the following three main contributions.

- 1) *A novel privacy-preserving and resource-balanced split Bert model [34] (termed SL-Bert)* is presented to address challenges 1) and 2). We offload the Transformer of the Bert to a high-performance VS, reducing client computation. Then, layered *feature filtering* and *irreversible intermediate activation* are applied to resist reverse reconstruction to ensure client-side privacy while balancing computing resources between VS and CS.

TABLE I
SUMMARY OF RELATED WORK IN LOG-BASED FAULT DIAGNOSIS

Objective	Proposal	Technology	Baselines	Reason
Fault Detection	LogAnomaly [35] DeepLog [36] LogRobust [37]	LSTM	Yes	Short-sequence inputs with controllable vector size minimize communication, suiting bandwidth-limited scenarios.
	HitAnomaly [38]	Transformer	No	The hierarchical input and output formats have low compatibility with our client-server split learning.
	NeuralLog [41]	Transformer	Yes	Upload data can be compressed into protected activations to assess split communication overhead.
	LogBert [42]	Transformer	No	The self-supervised, multi-task training objectives are incompatible with local fine-tuning.
	Wittkopp et al. [43]	Transformer	No	Decentralized point-to-point communication does not match our vendor-server aggregation baseline.
	FedLog [11]	TCN-ACNN	Yes	It is a native federated learning framework and a natural platform for researching personalization strategies.
	AdaLightLog [51]	Structural Adaptation	Yes	Federated log detector tailored to client heterogeneity.
	DistilLog [52]	Knowledge Distillation	Yes	Compact log detector for lightweight baseline comparison.
	FedDyn [53]	Dynamic Regularization	No	Full-model regularizer; no log-diagnosis architecture.
	FedNova [54]	Normalized Updates	No	Aggregation normalization only; no syslog diagnosis pipeline.
	FedProx [55]	Proximal Regularization	No	General FL optimizer; no syslog-specific diagnosis pipeline.
	FLUXLog [56]	MoE Routing	Yes	Federated log detector for cross-domain non-IID settings.
Root Cause Analysis	SCAFFOLD [57]	Control Variates	No	Optimization-level drift correction, not a log-diagnosis model.
	LogCluster [44] Cloud19 [45]	TF-IDF Word2Vec	Yes	Controllable communication overhead makes them suitable for our FSL scenario.
	LogBASA [46] LogKG [47]	System Function Path Diagram Knowledge Graph	No	Both of these works rely on graph structure, which is not available in our scenario.
	MoniLog [48]	Building a Log Stream	No	This work does not give the root cause of faults, so it is not applicable to our setting.
	SwissLog [49]	Combining Semantic Embedding with Temporal Embedding	No	This work supports root causes for two types of faults, which is inconsistent with our scenario.

- 2) An adaptive bidirectional FL-exponential moving average (EMA) loop for mitigating client drift (that is, FL-EMA) is proposed to address challenge 3). It utilizes forward FedAvg to perform multi-round parallel aggregation for a more accurate global model, and backward EMA employs a serial smoothing strategy, sequentially fine-tuning to effectively reduce local model bias. The bidirectional loop enhances the model's generalization and adaptability, mitigating client drift.
- 3) We introduce *FSLog*, an FSL-based framework tailored for privacy-protected and computation efficient syslog fault diagnosis with both fault detection and RCA in MaaS oriented IoT networks. Experimental results validated its effectiveness on two large-scale industrial datasets from Alibaba and China Mobile. *FSLog* achieves macro-averaged F1-scores (M-F1) of 0.95 and 0.96, outperforming the best baseline by 10.6% and 6.0%, respectively.

Article Organization: The structure of this article is as follows. In Section II, we introduce related work on fault diagnosis. We elaborate on *FSLog*'s framework in Section III. Subsequently, in Section IV, we describe the experimental setup and results for the IoT deployment. Afterward, we deploy *FSLog* in Docker and analyze specific fault cases in Section V. Finally, we conclude this article in Section VI.

II. RELATED WORKS

Since fault diagnosis comprises two tasks of the fault detection [11], [35], [36], [37], [38], [39], [40], [41], [42],

[43] and RCA [44], [45], [46], [47], [48], [49] in this article, we organize related work into two main research streams to better survey recent advances in fault diagnosis. Table I lists representative works in these two categories and why they were chosen as the baseline for our work or not.

A. Fault Detection

Numerous fault detection methods have been developed using architectures such as LSTM, RNN, and Transformer. We categorize these works into two classes based on their learning paradigm: traditional centralized learning and distributed collaborative learning, as detailed as follows.

1) *Traditional Centralized Learning:* They constitute the mainstream of log-based fault detection, where all training data are gathered at a single site for model optimization. These methods can be further divided by architecture into LSTM-based approaches for short log sequences and Transformer-based approaches for long log sequences.

a) *LSTM for short log sequences:* LogAnomaly [35] encodes logs into semantic vectors and uses Bi-LSTM for contextual anomaly scoring but is limited in modeling long sequences and sensitive to cross-client semantic shifts. DeepLog [36] maps logs to event sequences, using LSTM to detect sequence-level anomalies, yet struggles with template drift, sparse events, and long-range dependencies. LogRobust [37] employs word vectors for robust detection but faces challenges with long-term dependencies and resource constraints. All three methods operate on short sequences, making them suitable for real-time on-device detection and serving as strong baselines.

b) Transformer for long log sequences: Transformers [39], [41], [42], [43] enhance parallel computation and retain key contextual information, making them suitable for long-sequence tasks. Yang and Peng [50] proposed Bert-based architectures specifically designed for traffic classification, thereby demonstrating the successful adaptation of NLP Transformers to the cybersecurity domain. HitAnomaly [38] uses a hierarchical Transformer to encode log templates and parameters separately, improving hierarchical semantic representation and multigranularity temporal modeling, but relies on a specialized hierarchical input structure. NeuralLog [41] uses a Transformer with positional and semantic encodings to produce semantic vectors for detecting anomalies and complex fault patterns, aligning with our RCA. LogBert [42] employs a multilayer Transformer encoder with self-supervised learning on unlabeled data but incurs high communication and synchronization overhead due to multitask parallel training. Wittkopp and Acker [43] proposed a fully decentralized framework enabling knowledge sharing among Deeplog models without exchanging raw data or model parameters. Due to its decentralized topology, it complicates centralized server aggregation, unlike split-model approaches. FedLog [11] trains log models and updates to a central server to detect system faults. However, it does not perform further RCA on initially detected faults.

2) Distributed Collaborative Learning: Beyond traditional centralized learning, distributed collaborative learning specifically targets data heterogeneity in federated environments. Existing drift-mitigation research generally spans four theoretical directions.

a) Optimization constraints: FedAvg [5] aggregates updates via simple averaging, risking divergence under non-IID conditions without explicit drift constraints. FedProx [55] restricts parameter deviations via proximal regularization, but its data-dependent residual penalty severely weakens control efficacy under extreme heterogeneity. FedDyn [53] aligns local objectives using dynamic regularization, yet its reliance on precise gradient tracking inversely amplifies drift via stale biases under skewed distributions.

b) Gradient correction: SCAFFOLD [57] corrects drift directions using control variates. Practically, estimation biases induced by local data instability directly translate into correction misalignment. FedNova [54] normalizes local update steps to resolve objective inconsistency. However, this server-side weight adjustment completely lacks direct drift suppression during actual local training.

c) Architecture partitioning: FLUXLog [56] mitigates gradient conflicts via mixture-of-experts (MoE) routing. Relying on standard FedAvg aggregation, it inherently fails to rectify parameter drift across highly divergent clients.

d) Aggregation strategy: AdaLightLog [51] restricts structural adaptation to individual clients, entirely omitting network-wide consistency constraints. DistilLog [52] achieves model compression via knowledge distillation, risking the propagation of teacher biases by bypassing cross-client gradient alignment. FedLog [11] inherits FedAvg for federated syslog detection, completely lacking targeted drift control against intraclient component-level heterogeneity.

Fundamentally, these drift constraints universally rely on data-dependent intermediate variables, causing their theoretical efficacy to inevitably decay as non-IID severity increases. Crucially, they universally assume clients host the complete model. Once the backbone is offloaded in SL architectures, local regularizers cannot cover the primary parameter space. This exposes a critical structural incompatibility, fundamentally necessitating a parameter-space drift control mechanism free of data-dependent residuals.

Recent studies have investigated federated split learning in IoT and Industrial IoT environments. DFL [58] addresses both data and resource heterogeneity by resource-aware split computing. It is designed as a general FSL protocol and does not model syslog sequences or RCA. FedSplitX [59] further supports computationally constrained heterogeneous clients through multiple partition points and auxiliary networks, yet it is not tailored to syslog semantics or RCA. In parallel, FL has been adopted for log fault detection. FedSeq [60] studies personalized federated anomaly detection for discrete event sequences, and FedLAD [61] provides a modular testbed for evaluating log anomaly detection models under federated settings. These works demonstrate that federated log fault detection is an active research direction. However, they mainly focus on fault detection. They do not jointly address SL-based Transformer offloading, privacy-preserving intermediate representations, component-level non-IID syslog grouping, and multiclass RCA in MaaS-oriented IoT networks.

B. Root-Cause Analysis

Initial fault detection cannot localize the fault source or type. Therefore, we perform RCA on detection to precisely identify the fault source and guide O&M staff. We categorize existing log-based RCA [44], [45], [46], [47], [48], [49] methods into three classes.

1) Machine Learning: LogCluster [44] employs TF-IDF [62] to convert log sequences into weighted event count vectors, and applies hierarchical clustering for failure type identification. It depends on word frequency statistics, thus lacking in capturing deep semantics and long-range temporal dependencies. Cloud19 [45] uses Word2Vec [63] for vectorization and classification for failure type annotation, with embedding quality heavily reliant on training data. Both LogCluster and Cloud19 generate low-dimensional, structured features [64]. Their controllable communication overhead [65], [66] makes them suitable for FSL scenarios.

2) Graph Structure: LogBASA [46] uses a pretrained Bert model to sense semantic information about the log event sequence. When the model detects the fault, the system's functional path diagram can improve the efficiency of the diagnosis. The diagram may be unavailable in real internal networks. LogKG [47] fully extracts entities and relationships from logs and mines multifield information through knowledge graphs (KGs). FOLR [47] is used to extract fault-related patterns and train the fault diagnosis model to determine the RCA. KG construction and cross-domain information integration incur high overhead. Both methods [46], [47] rely on graphs, which are not available in our scenario.

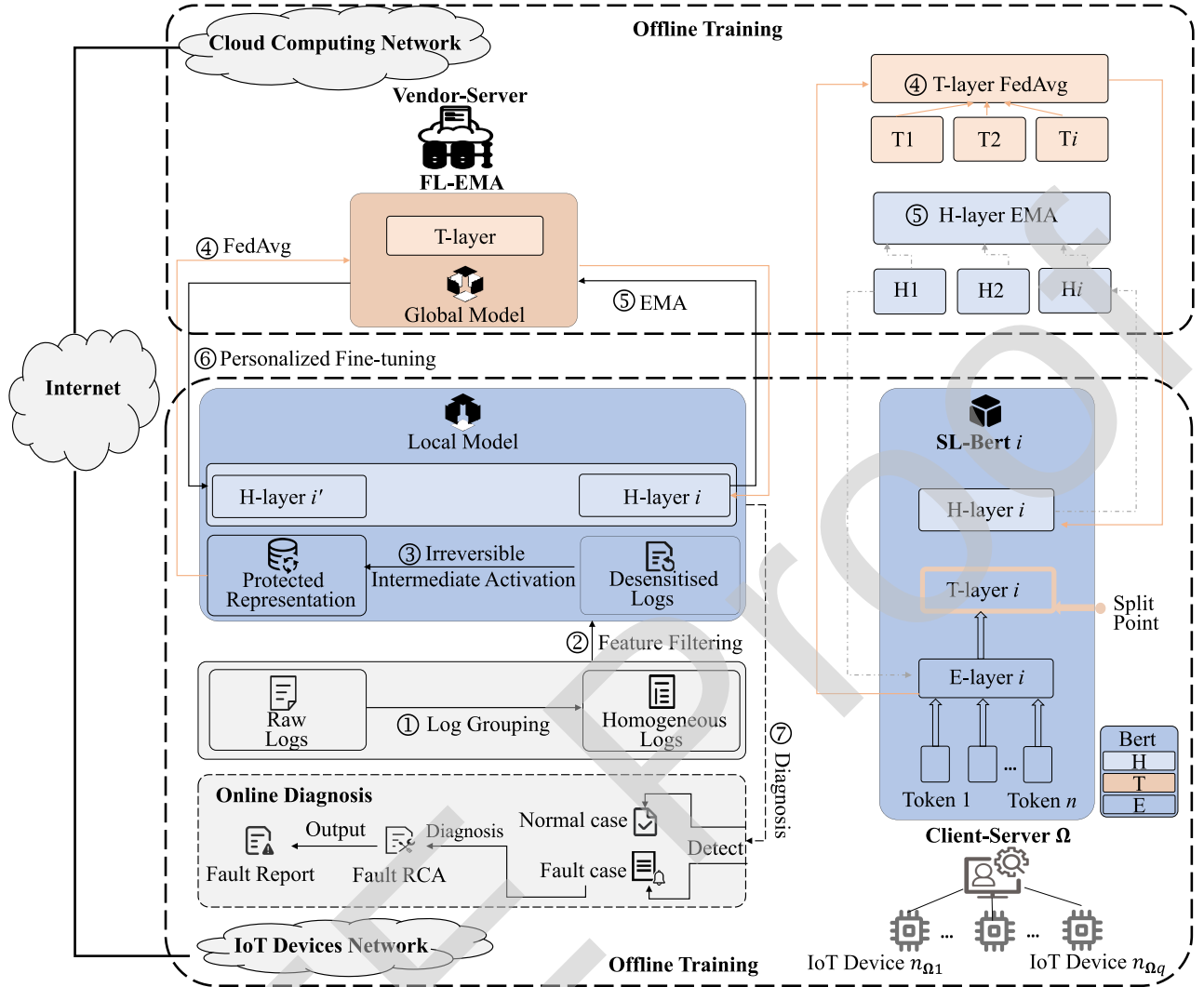


Fig. 2. *FSLog* framework with three-layer split Bert across VS and CS in IoT networks.

3) *Deep Learning*: *MoniLog* [48] builds log-stream models to detect anomalous sequences and quantify anomalies, outputting anomaly streams with severity levels. They focus on anomaly flows and typically do not produce explicit RCA labels, failing to meet case-level RCA needs. *SwissLog* [49] combines semantic embeddings with temporal embeddings to diagnose faults that alter log order or intervals. It only covers faults manifested by sequence order or timing changes, limiting its applicability. Regarding model optimization and accuracy, the work of Eldakhly [67] illustrates the potential of deep learning models to improve classification rates in FL, thereby establishing a basis for concentrated research efforts on deep architectures. Collectively, these findings indicate that machine and deep-learning-based faulty detection offer an effective solution to the problem of false positives (FPs) in IoT systems.

In summary, traditional FL models have predominantly prioritized model accuracy at the expense of accounting for unreliable node involvement, noisy gradient updates, and energy disparities across the IoT network. Few studies have explored the deployment of FSL log-based fault diagnosis, most of which are fault detection efforts that fail to

fully mine RCA. Therefore, we need to fill the gap in the field.

III. FSLOG FRAMEWORK

A. Workflow of *FSLog*

We developed the *FSLog* framework for client-side and server-side collaborative diagnosis. As shown in Fig. 2, the IoT network adopts a three-tier hierarchical architecture, consisting of a VS, multiple gateways, and a large number of IoT O&M and users. Each IoT network serves a large number of IoT users (i.e., various IoT devices). This is achieved by gateways collecting data from IoT users within their coverage and processing it using their equipped CS. The IoT devices network and cloud computing network engage in communication over Internet. According to task complexity, two processes need to be carried out. Computational tasks may be processed on the CS through the gateway or offloaded to the VS for execution. In other words, lightweight tasks are assigned to local IoT devices, and high-load tasks are primarily handled by the VS. For simple tasks, *FSLog* merely performs lightweight offline training and online diagnosis on

the client, enabling rapid preliminary alerts. However, for complex tasks that require privacy protection, we utilize the **SL-Bert** method with *multilayer privacy-preserving chains* (② and ③) that the compute-intensive Transformer backbone is hosted on the VS to form a protected environment. The client-side performs log preprocessing and online diagnosis by applying the component-driven log grouping ①, feature filtering ②, and irreversible intermediate activation ③ to the raw log, from which a protected intermediate representation is formed and then uploaded to the VS. Subsequently, simultaneous semi-supervised offline training is performed using the **FL-EMA** method across both the CS and VS. On the server-side, it executes *FedAvg with parallel forward aggregation* ④ and *EMA-driven sequential backward personalization* ⑤, respectively. Following this, on the client-side, the local model adopts *personalized fine-tuning* ⑥ by merging the head with parameters transferred from the VS after training. *FSLog* achieves accurate RCA in IoT systems based on Docker in a “fast first, then accurate” manner.

B. Component-Driven Log Grouping

In the IoT devices network of large-scale parallel systems, preprocessing and structuring of syslogs are prioritized. Then, these logs are grouped using their structural and semantic features to ensure high similarity within each group. More concretely, the CS is tasked with collecting data from IoT devices (end devices) and partitioning the data points into different clusters, such that points within a cluster exhibit similar features and are distinct from those in other clusters.

1) *Lightweight Edge Preprocessing*: We first perform basic data cleaning (e.g., timestamp normalization, deduplication, and encoding standardization), and use Drain to convert the raw log message l into $\langle \text{template}, \text{params} \rangle$, thereby obtaining the processed log, which is represented as l' . This process can reduce text sparsity and retain structured semantic information.

2) *Homogenized Local Log*: We need to allocate logs of the same type to the same server model in log order to distinguish the non-IID data from different IoT devices. To this end, each client performs a coarse-to-fine partition by a hierarchical field-attribute key, and the whole procedure is summarized as a deterministic pipeline in Algorithm 1. First, the logs are initially filtered by server model. The grouping key follows the priority chain $\kappa(v) = \langle \text{SM}(v), \text{SN}(v), \text{TID}(v) \rangle$, where $\text{SM}(\cdot)$, $\text{SN}(\cdot)$, and $\text{TID}(\cdot)$ denote the server model, the server number, and the Drain-parsed template ID of a log entry v , respectively. The priority is ordered from coarse physical-isolation attributes to fine semantic attributes, since the server model determines the hardware-semantic domain that dominates the non-IID boundary, the server number identifies the physical node, and the template ID finally captures the intracomponent semantic pattern. Subsequently, the logs are grouped based on the template ID within the time window, grouping semantically similar logs into the same group, thereby forming a local homogeneous dataset. In other words, within each sliding window, entries that share an identical key $\kappa(v)$ are aggregated into one homogeneous sequence.

To remove assignment ambiguity during this grouping, we adopt a deterministic conflict-resolution rule, so that each log

Algorithm 1 Component-Driven Log Grouping

Require: Parsed logs $L = \{(\text{template}, \text{params}, \text{attrs})\}$, attribute priority $P = (\text{SM}, \text{SN}, \text{TID})$, window size w , minimum group size $m_{\min}$

Ensure: Homogeneous local log sequences $\mathcal{X}$

- 1 Initialize $\mathcal{G} \leftarrow \emptyset$, residual buffer $\mathcal{B} \leftarrow \emptyset$, output $\mathcal{X} \leftarrow \emptyset$
- 2 **for** each log $v \in L$ **do**
- 3 $\kappa(v) \leftarrow$ highest-priority non-missing key of v along P
- 4 **if** all attributes of v are missing **then**
- 5 $\kappa(v) \leftarrow \arg \max_g \cos(\text{emb}(v.\text{template}), \text{centroid}(g))$
- 6 **end if**
- 7 **if** multiple candidate groups tie on $\kappa(v)$ **then**
- 8 break tie by priority chain, then minimum temporal distance, then smallest index
- 9 **end if**
- 10 assign v to the unique group $g_{\kappa(v)}$
- 11 **end for**
- 12 **for** each group $G \in \mathcal{G}$ **do**
- 13 **if** $|G| \geq w$ **then**
- 14 slice G chronologically into $x^{(i)} = (v_i, \dots, v_{i+w-1})$; add to $\mathcal{X}$
- 15 **else if** $m_{\min} \leq |G| < w$ **then**
- 16 pad G to length w with PAD tokens and build a valid-token mask; add to $\mathcal{X}$
- 17 // imbalance absorbed by weighted loss, Eq.(36)
- 18 **else**
- 19 find nearest group G' with same SM by cosine similarity of template centroids
- 20 **if** G' exists **then**
- 21 merge G into G'
- 22 **else**
- 23 $\mathcal{B} \leftarrow \mathcal{B} \cup \{G\}$ // no pseudo-label offline; low-context evidence online
- 24 **end if**
- 25 **end if**
- 26 **end for**
- 27 **return:** $\mathcal{X}$

sequence is mapped to exactly one group. When different attributes of a log point to different candidate groups, the higher priority attribute prevails along the key chain. When a high-priority attribute is missing, the procedure falls back to the next-priority attribute. When all key attributes are missing, the log is assigned to the semantically nearest group by the cosine similarity between its Drain template embedding and the group centroid. To guarantee that independent implementations reproduce identical groupings, the remaining ties are broken in a fixed order. We first compare the attributes one by one along the priority chain. We then select the group with the minimum temporal distance to the current log. We finally select the group with the smallest index.

Specifically, we first group each client's logs sequentially according to the same field attribute (such as server model, server number, and event ID). Then, the grouped logs are organized as an ordered stream $v = (v_1, v_2, \dots, v_n)$. Using a sliding window of size w , the i th log sequence is defined as

$x^{(i)} = (v_i, v_{i+1}, \dots, v_{i+w-1})$. The partial log sequence is shown at the far-right end node in Fig. 1. For groups whose size is insufficient to fill a complete window, we further introduce a minimum group size $m_{\min}$ to control sequence generation. A group with at least w logs is sliced directly, whereas a group below w yet no less than $m_{\min}$ is padded to length w with a valid-token mask that excludes the padded positions from attention and loss, with its residual imbalance absorbed by the weighted cross-entropy loss in (36). A group below $m_{\min}$ is merged into the nearest same-server-model group by template-centroid similarity, and otherwise retained in a client-side residual buffer that yields no pseudo-label offline yet still serves as low-context evidence during online diagnosis, so that rare fault categories remain trainable without discarding diagnostically valuable sequences.

C. Split Bert Model With Privacy-Preserving Chains

To enhance the data security of the CE, this method conducts feature filtering at the embedding layer (E-layer), then employs activation encoding at the segmentation point of the model, and finally achieves effective intermediate representation protection between the E-layer and the T-layer. To balance computation and privacy, the system employs a split Bert, which is used to solve the pretraining task on constrained devices when the CS does not have enough GPU compute resources. For a given input log sequence $x^{(i)}$, the output of the split Bert model $F(x^{(i)})$ is defined in (1). The pretraining task is denoted as $F(x^{(i)})$. The output of the Bert model after splitting is denoted as defined in the following equation:

$$F(x^{(i)}) = W_H^u (W_T^u (W_E^u x^{(i)})). \quad (1)$$

Here, the E-layer W_E^u and the personalized H-layer W_H^u reside on the CS. The entire Transformer backbone W_T^u is offloaded to the VS. During training, the CS transmits protected intermediate representations and cut-layer gradients ($\partial L / \partial t f_{Tensor}$) to the VS, and receives the Transformer output for local H-layer computation.

1) *Feature Filtering*: To accommodate client-side computational limits and minimize sensitive data leakage, the system embeds templates and parameterized messages instead of processing raw texts. It then applies a dual-stage feature filtering mechanism. Field-level filtering removes sensitive tokens prior to embedding construction, while importance filtering statistically evaluates tokens to eliminate high-risk features. This approach secures local embedding generation. It strictly retains essential semantics on the client side and mitigates the risk of reverse message reconstruction, thereby enhancing overall data security. More specifically, to capture sensitive words during importance filtering, the system employs Kendall's correlation coefficient. Before computing this correlation, the CS locally assigns a sensitivity indicator to each token. Let q_j be the j th token after Drain parsing and f_j its source field (e.g., server model, timestamp, or event ID). The indicator is generated by a local detector $S(\cdot)$, as shown in the following equation:

$$b_j = S(q_j, f_j) \quad b_j \in [0, 1]. \quad (2)$$

This detector combines field-priority rules, regular-expression patterns (e.g., IP/MAC, user IDs, and region/rack tags), and

a sensitivity dictionary. Tokens matching predefined patterns receive high sensitivity scores ($b_j \rightarrow 1$), while task-relevant fault semantics are assigned low scores and retained. This detection executes entirely locally, ensuring neither the dictionary nor the b_j indicators are transmitted to the VS. Subsequently, the system calculates Kendall's correlation τ to evaluate the structural relationship between token occurrence frequency and sensitivity, as expressed in the following equation:

$$\tau = \frac{2}{n(n-1)} \sum_{1 \leq j < k \leq n} \text{sgn}(a_j - a_k) \text{sgn}(b_j - b_k). \quad (3)$$

Here, a_j and a_k denote the occurrence frequencies of tokens q_j and q_k ($1 \leq j < k \leq n$), respectively. b_j and b_k represent their locally assigned sensitivity indicators. A token with a correlation magnitude $|\tau| \geq \tau_0$ ($= 0.5$) is treated as sensitivity-associated and is strictly retained on the CS. The remaining desensitized content is then processed further. Ultimately, only the filtered embedding and the irreversibly encoded activation are sent to the VS. To intuitively illustrate the practical application of this correlation-based filtering mechanism, we provide a concrete example using a real-world log message, as detailed in Example 1.

Example 1: Consider the sample log message shown in Fig. 1: "SM5 Server_16 2020-10-09 21:05:03 69b36af7 Memory #0x48 | Configuration Error | Deasserted." This log is processed within a sliding window of $w = 6$ consecutive messages, which is consistent with the default window size selected in Section IV-D. Note that (3) adopts the τ - a form of Kendall's correlation, in which tied pairs contribute zero through the sign function.

- 1) *Variable Instantiation*: For the token "Server_16," suppose its occurrence frequencies across the six positions are $a = (2, 1, 3, 1, 2, 1)$, and the co-occurring sensitivity indicators are $b = (1, 0, 1, 0, 1, 0)$.
- 2) *Correlation Evaluation*: Substituting these vectors into (3) over all $\binom{6}{2} = 15$ pairs, nine pairs yield $\text{sgn}(a_j - a_k) \text{sgn}(b_j - b_k) = +1$ and the remaining six pairs contain ties and contribute 0. The correlation is therefore $\tau = 0.6$.
- 3) *Filtering Execution*: Since $|\tau| = 0.6 > \tau_0 = 0.5$, the token "Server_16" exhibits a strong dependency on sensitive attributes and is strictly retained on the CS, together with the explicit timestamp, the event hash "69b36af7," and the hardware slot identifier "#0x48." In contrast, essential fault-semantic tokens such as "Memory," "Configuration Error," and "Deasserted" yield a negligible correlation ($\tau \approx 0$) and bypass the filter.
- 4) *Desensitization Outcome*: After applying this mechanism, the raw message is structurally desensitized into a safe template: "[SM] [SERVER_ID] [TIME] [EVENT_ID] Memory #[SLOT] | Configuration Error | Deasserted."

This transformation securely eliminates device-identifying context before the data is embedded and encoded,

comprehensively preserving the fault semantics required for downstream RCA while strictly bounding privacy leakage.

2) *Irreversible Intermediate Activation*: To protect the intermediate representation at the split point between the E-layer on the CS and the T-layer on the VS, the CS applies a lightweight bottleneck activation encoder E_θ to the filtered E-layer output. Given the i th log sequence $x^{(i)}$ on client u , the filtered embedding representation is first obtained via the following equation:

$$A_u^{(i)} = \text{Filter}_\tau(W_E^u(x^{(i)})) \quad (4)$$

where $A_u^{(i)} \in \mathbb{R}^{n \times d_e}$, n denotes the log sequence length, and d_e denotes the embedding dimension. The irreversible activation encoder then maps $A_u^{(i)}$ into a low-dimensional protected representation, as defined in the following equation:

$$Z_u^{(i)} = E_\theta(A_u^{(i)}) = E_\theta(\text{Filter}_\tau(W_E^u(x^{(i)}))) \quad (5)$$

where $Z_u^{(i)} \in \mathbb{R}^{n \times d_z}$ and $d_z < d_e$. During this transformation, the sequence length n is preserved, while only the feature dimension is compressed. E_θ is implemented as a two-layer bottleneck network parameterized by $\theta = \{W_1, W_2, b_1, b_2\}$. Its forward mapping is defined as shown in the following equation:

$$Z_u^{(i)} = \text{LN}(\text{Dropout}(\text{GELU}(A_u^{(i)}W_1 + b_1))W_2 + b_2) \quad (6)$$

where $W_1 \in \mathbb{R}^{d_e \times d_b}$, $W_2 \in \mathbb{R}^{d_b \times d_z}$, and d_b represents the bottleneck hidden dimension that enforces the information bottleneck via severe dimensionality reduction. GELU serves as the nonlinear activation function that introduces probabilistic gating-based structural deformation to resist inverse approximation. Dropout stochastically masks intermediate neurons to further amplify inversion difficulty, and LN denotes the layer normalization to stabilize the protected representation. This bottleneck structure performs a lossy and noninjective transformation, making the uploaded representation resistant to reconstruction. Since high-sensitivity tokens have already been removed by Kendall's τ -based feature filtering, the remaining embedding mainly contains desensitized semantic information with reduced reconstruction value. The combination of feature filtering and irreversible encoding forms a two-layer privacy-preserving chain. The first layer suppresses sensitive information at the token level, while the second layer obfuscates the filtered representation at the activation level. Together, they substantially reduce the risk of reconstructing the original syslog content and inferring sensitive attributes from the protected representation uploaded to the VS.

D. Synergistic Bidirectional FL-EMA Loop

The mechanism forms a bidirectional loop at the VS and CS. The CS retains embeddings and personalized heads while the VS hosts the Transformer backbone, so only protected activations and cut-layer gradients are transmitted, enabling integration of secure aggregation. During FL-EMA training, the embedding layer W_E^u is frozen and is used only for local forward representation generation. In the forward phase, the CS uploads protected intermediate representations, and the

VS updates the Transformer replicas in parallel and aggregates parameters via FedAvg to obtain a global Transformer backbone. In the backward phase, the VS performs sequential EMA on the aggregated Transformer parameters across clients. Meanwhile, each CS independently applies a local temporal EMA to smooth its personalized H-layer. The two EMA operations are decoupled and do not require cross-boundary parameter transmission. This forward aggregation and backward personalization process repeats cyclically until the model converges. It is noteworthy that for the two training strategies, the VS maintains different numbers of Transformer blocks. In parallel training, the VS stores N Transformer replicas, one per client. After all clients complete local training, the VS aggregates the N CS Transformer replicas into a global Transformer backbone and shares it with all clients. In the sequential training, the VS needs only a single Transformer block, and each client updates that Transformer block.

1) *FedAvg With Parallel Forward Aggregation*: Each client calculates the embedding representation locally and applies feature filtering to this representation. On the VS side, using (6) as input, the forward computation and gradient update are completed in parallel on the Transformer backbone of the T-layer. In each training epoch, we store the corresponding T-layer for each client on the VS to maintain the consistency of the model in each training period

$$F_{u,r}(x^{(i)}) = H_{W_H^r}(T_{W_E^u}(Z_u^{(i)})) \quad (7)$$

In the r th round, after all clients complete forward propagation and gradient updates, the VS aggregates the Transformer parameters of all clients based on sample weights, as shown in (8). Here, $r = 0, 1, \dots, R - 1$ denotes the current communication round, $r + 1$ denotes the next communication-round state obtained after completing the current FedAvg and EMA updates, and R denotes the total number of communication rounds. Therefore, parameters indexed by T^R or H^R correspond to the final trained model

$$W_{T^{r+1}}^{\text{agg}} \leftarrow \sum_{j=1}^N \frac{|D_{u_j}|}{|D|} W_{T^r}^{u_j}, \quad |D| = \sum_{j=1}^N |D_{u_j}| \quad (8)$$

where $|D_{u_j}|$ denotes the data volume of client u_j , $|D|$ is the total data volume, N denotes the number of CSs, and $W_{T^r}^{u_j}$ is the Transformer parameter of client u_j after local training in round r . Only W_T^u participates in FedAvg aggregation. The frozen E-layer W_E^u and the personalized H-layer W_H^u are retained locally on each CS and do not participate in server-side aggregation.

2) *EMA-Driven Sequential Backward Personalization*: The backward personalization of FL-EMA consists of two decoupled EMA operations, executed independently on VS and CS. *Server-side Sequential Transformer EMA*. After FedAvg aggregation, the VS uses $W_{T^{r+1}}^{\text{agg}}$ as the initial EMA state and processes each CS u_j in sequence. We denote the server-side sequential EMA step index as $s_j (j = 0, 1, \dots, N)$ to distinguish it from the CS index u_j . As presented in the following equation, the aggregated Transformer parameters serve as the starting point for the sequential EMA process, i.e., step s_0 :

$$W_{T^{r+1}}^{\text{ema}(s_0)} \leftarrow W_{T^{r+1}}^{\text{agg}} \quad (9)$$

Then, as described in (10), the VS sequentially incorporates each client's locally optimized Transformer parameters into the EMA state. Here, $W_{Tr}^{u_j*}$ denotes the Transformer parameters of client u_j after local fine-tuning in round r , where the superscript $*$ indicates that these parameters have been updated through local training rather than remaining in their initial state. At step s_j , the VS incorporates the parameters of client u_j into the EMA state

$$W_{Tr+1}^{\text{ema}(s_j)} \leftarrow \delta_T W_{Tr+1}^{\text{ema}(s_{j-1})} + (1 - \delta_T) W_{Tr}^{u_j*}. \quad (10)$$

After all N ($j = 1, 2, \dots, N$) clients have been processed, i.e., the VS completes step s_N , the final EMA result is obtained as shown in the following equation:

$$W_{Tr+1}^{\text{ema}(s)} \leftarrow W_{Tr+1}^{\text{ema}(s_N)}. \quad (11)$$

Here, $\delta_T \in [0, 1]$ is the server-side EMA weighting coefficient. A larger δ_T places more weight on the accumulated knowledge from previously processed clients. A smaller δ_T allows the current client's contribution to dominate. This process enables the Transformer to incrementally absorb personalized updates from all clients. No additional parameter uploads from the CS are required, as all Transformer parameters already reside on the VS. We use the fixed coefficient δ_T in (10) to keep the effective aggregation weights consistent with the subsequent convergence analysis.

Client-Side Local Head EMA: After completing local training in each round, each client u independently smooths its personalized H-layer across rounds. As shown in the following equation, the smoothed H-layer parameter $\bar{W}_{H^{r+1}}^{\text{ema}(u)}$ is computed by blending the previous smoothed result $\bar{W}_{H^r}^{\text{ema}(u)}$ with the newly trained H-layer parameter $W_{H^r}^{u*}$ of the current round:

$$\bar{W}_{H^{r+1}}^{\text{ema}(u)} \leftarrow \delta_H^u \bar{W}_{H^r}^{\text{ema}(u)} + (1 - \delta_H^u) W_{H^r}^{u*}. \quad (12)$$

A larger δ_H^u preserves more historical knowledge and prevents abrupt shifts in the classification boundary. This operation is executed entirely on the CS without transmitting any parameters to the VS. The server-side coefficient δ_T and the client-side coefficient δ_H^u serve distinct purposes. The server-side EMA fuses cross-client knowledge for the shared Transformer along the server-side step dimension s_j . The client-side EMA stabilizes per-client personalized classification along the round dimension r . After both EMA operations, the personalized model of the client u is defined by the parameter set shown in the following equation:

$$\Theta_u^{r+1} \leftarrow \{W_E^u, E_\theta, W_{Tr+1}^{\text{ema}(s)}, \bar{W}_{H^{r+1}}^{\text{ema}(u)}\}. \quad (13)$$

Based on the above parameter set, the personalized model of client u performs inference on input x as specified in (14). This notation separates the model function $\mathcal{M}_u$ from its parameter set Θ_u

$$\begin{aligned} \mathcal{M}_u^{r+1}(x; \Theta_u^{r+1}) \\ = H_{\bar{W}_{H^{r+1}}^{\text{ema}(u)}} \left(T_{W_{Tr+1}^{\text{ema}(s)}} (E_\theta (\text{Filter}_\tau (W_E^u(x)))) \right). \end{aligned} \quad (14)$$

Here, W_E^u is the frozen local embedding parameter retained on the CS, E_θ is the irreversible activation encoder, $W_{Tr+1}^{\text{ema}(s)}$

is the Transformer backbone obtained through FedAvg aggregation followed by sequential EMA on the VS, and $\bar{W}_{H^{r+1}}^{\text{ema}(u)}$ is the locally smoothed personalized head. The operator Filter_τ is not a trainable parameter but remains part of the privacy-preserving inference path. This equation describes the computational flow of the assembled personalized model during inference, where the input x is sequentially processed through the frozen embedding layer, the feature-filtering operator, the irreversible activation encoder E_θ , the EMA-smoothed Transformer, and the EMA-smoothed head to produce the final prediction. The final personalized model after training is obtained by setting $r + 1 = R$ in (13) and (14). The convergence properties of the above bidirectional loop under nonconvex conditions are analyzed in Section III-D3.

3) Theoretical Convergence Analysis: This section establishes a formal convergence guarantee for the bidirectional FL-EMA loop under nonconvex optimization. Let $f(W) = \sum_{u=1}^N p_u f_u(W)$ with $p_u = |D_u|/|D|$ be the global objective. The analysis builds upon the notation of the Transformer parameter updates in (8)–(11) and introduces necessary auxiliary quantities. Three standard regularity conditions for federated nonconvex analysis [68] are adopted.

Assumption 1 (L-Smoothness): Each local loss f_u possesses an L -Lipschitz continuous gradient, i.e., for all W, W' , the gradient deviation is bounded as shown in the following equation:

$$\|\nabla f_u(W) - \nabla f_u(W')\| \leq L \|W - W'\|. \quad (15)$$

Assumption 2 (Bounded Local Variance): The stochastic gradient $g_u(W; \xi)$ is an unbiased estimator of $\nabla f_u(W)$ with uniformly bounded variance. The variance satisfies the following equation:

$$\mathbb{E}_\xi \|g_u(W; \xi) - \nabla f_u(W)\|^2 \leq \sigma^2 \quad \forall u, W. \quad (16)$$

Assumption 3 (Bounded Gradient Dissimilarity): The cross-client gradient shift induced by non-IID data distributions is bounded. The gradient dissimilarity satisfies the following equation:

$$\sum_{u=1}^N p_u \|\nabla f_u(W) - \nabla f(W)\|^2 \leq \zeta^2 \quad \forall W. \quad (17)$$

Assumptions 1–3 constitute the standard regularity conditions for nonconvex federated optimization and, respectively, characterize the local curvature of the loss surface, the per-client sampling noise level, and the degree of data distribution shift across clients. Building upon these assumptions, we first characterize the equivalent aggregation structure of the FL-EMA update.

Lemma 1 (Effective Aggregation Weights): The FL-EMA update in round r is equivalent to a single-pass weighted aggregation of client parameters. The update result is obtained as follows:

$$W_{Tr+1}^{\text{ema}} = \sum_{j=1}^N \gamma_{u_j} W_{Tr}^{u_j*} \quad (18)$$

where the effective weight γ_{u_j} is jointly determined by the data proportion from FedAvg and the sequential position from EMA. γ_{u_j} is computed as follows:

$$\gamma_{u_j} = \delta_T^N p_{u_j} + (1 - \delta_T) \delta_T^{N-j}. \quad (19)$$

Proof: We unroll the server-side EMA recursion in (10) starting from the initialization $W_{T+1}^{\text{ema}(s_0)} = W_{T+1}^{\text{agg}}$ in (9). For the first step, $W_{T+1}^{\text{ema}(s_1)} = \delta_T W_{T+1}^{\text{agg}} + (1 - \delta_T) W_{T+1}^{u_1*}$. Applying (10) once more gives $W_{T+1}^{\text{ema}(s_2)} = \delta_T^2 W_{T+1}^{\text{agg}} + (1 - \delta_T) \delta_T W_{T+1}^{u_1*} + (1 - \delta_T) W_{T+1}^{u_2*}$. Proceeding inductively through step s_N , the contribution of the initial state is scaled by δ_T^N , while the locally optimized parameters of client u are weighted by $(1 - \delta_T) \delta_T^{N-j}$. This produces the closed-form expression, as shown in the following equation:

$$W_{T+1}^{\text{ema}(s_N)} = \delta_T^N W_{T+1}^{\text{agg}} + (1 - \delta_T) \sum_{j=1}^N \delta_T^{N-j} W_{T+1}^{u_j*}. \quad (20)$$

From (8), we have $W_{T+1}^{\text{agg}} = \sum_{j=1}^N p_{u_j} W_{T+1}^{u_j*}$. We substitute this result into (20) and collect terms by client index. This operation produces (18) and (19). The effective weights are normalized, which follows by summing the geometric series and adding the FedAvg contribution, as shown in the following equation:

$$\begin{aligned} \sum_{j=1}^N \gamma_{u_j} &= \delta_T^N \sum_{j=1}^N p_{u_j} + (1 - \delta_T) \sum_{j=1}^N \delta_T^{N-j} \\ &= \delta_T^N + (1 - \delta_T) \frac{1 - \delta_T^N}{1 - \delta_T} = 1. \end{aligned} \quad (21)$$

The boundary case $\delta_T = 1$ follows by continuity and reduces to standard FedAvg where $\gamma_{u_j} = p_{u_j}$.

This completes the proof. ■

Lemma 1 unifies the forward aggregation and the backward smoothing of FL-EMA into a single reweighted summation. The coefficient δ_T acts as a continuous interpolation parameter: $\delta_T \rightarrow 1$ recovers standard FedAvg ($\gamma_{u_j} \rightarrow p_{u_j}$), while decreasing δ_T progressively amplifies the contribution of later-processed clients and thereby strengthening personalization. To quantify the impact of this weight shift on convergence, we introduce the following discrepancy measure.

Definition 1 (Weight Discrepancy): The weight discrepancy measure $\Lambda(\delta_T, N)$ between the FL-EMA effective weights and the data-proportional weights is defined as follows:

$$\begin{aligned} \Lambda(\delta_T, N) &= \sum_{j=1}^N (\gamma_{u_j} - p_{u_j})^2 = (1 - \delta_T^N)^2 \|q - p\|_2^2 \\ &\leq 2(1 - \delta_T^N)^2 \end{aligned} \quad (22)$$

$$q_{u_j} = \frac{(1 - \delta_T) \delta_T^{N-j}}{1 - \delta_T^N}. \quad (23)$$

Here, $p = \{p_{u_j}\}$ with $p_{u_j} = |D_{u_j}|/|D|$ denotes the data-proportional weight vector, and $q = \{q_{u_j}\}$ denotes the normalized EMA positional weight vector defined in (23). Both p and q are nonnegative and sum to one. $\Lambda(\delta_T, N)$ satisfies $\Lambda(1, N) = 0$ and is uniformly bounded by $\Lambda(\delta_T, N) \leq 2(1 - \delta_T^N)^2$. We next establish a quantitative bound on the parameter drift accumulated during local training.

Lemma 2 (Local Drift Bound): Under Assumptions 1 and 2 with $\eta \leq 1/(4LE)$, after E local SGD steps, the parameter drift of client u is bounded. The average drift satisfies the following inequality:

$$\frac{1}{E} \sum_{e=0}^{E-1} \mathbb{E} \|W_r^{u,e} - W_r\|^2 \leq 4\eta^2 E (\sigma^2 + \|\nabla f_u(W_r)\|^2). \quad (24)$$

Proof: The local update rule is $W_r^{u,e+1} = W_r^{u,e} - \eta g_u(W_r^{u,e}; \xi)$. We apply this rule recursively to obtain $W_r^{u,e} - W_r = -\eta \sum_{t=0}^{e-1} g_u^{(t)}$. We take the squared norm in expectation on both sides. The unbiasedness and variance bound in Assumption 2 control the noise term. The L -smoothness condition under $\eta \leq 1/(4LE)$ bounds the gradient deviation from initial values. We then apply Young's inequality and average over e . This yields (24). ■

Combining Lemmas 1 and 2 with Definition 1, we are now in a position to state the main convergence result.

Theorem 1 (Nonconvex Convergence Rate of FL-EMA): We consider Assumptions 1–3 and set $\eta \leq 1/(4LE)$ and define $\Gamma = \sum_u \gamma_u^2$ and $G_f^2 = \sup_W \sum_u \|\nabla f_u(W)\|^2$. After R communication rounds, the average squared gradient norm of global Transformer parameters satisfies the bound in (25), as shown at the bottom of the page.

Proof: The proof proceeds in four steps.

Step 1 (Per-Round Descent): By Lemma 1, the FL-EMA update can be cast as a pseudo-gradient step $W_{r+1} = W_r - \eta E \bar{d}_r$, where the aggregated gradient direction is $\bar{d}_r = 1/E \sum_u \gamma_u \sum_{e=0}^{E-1} g_u(W_r^{u,e}; \xi)$. Applying the quadratic upper bound from Assumption 1, the per-round descent inequality is obtained as shown in the following equation:

$$\begin{aligned} f(W_{r+1}) &\leq f(W_r) - \eta E \langle \nabla f(W_r), \mathbb{E}[\bar{d}_r] \rangle \\ &\quad + \frac{L\eta^2 E^2}{2} \mathbb{E} \|\bar{d}_r\|^2. \end{aligned} \quad (26)$$

Step 2 (Bias Decomposition): At the initial local iterate $e = 0$ where $W_r^{u_0} = W_r$, the expected direction can be decomposed. It takes the form as shown in the following equation:

$$\sum_u \gamma_u \nabla f_u(W_r) = \nabla f(W_r) + b_r \quad (27)$$

where $b_r = \sum_u (\gamma_u - p_u) \nabla f_u(W_r)$ denotes the EMA-induced bias. By the Cauchy-Schwarz inequality, its squared norm is controlled by the weight discrepancy $\Lambda(\delta_T, N)$ in (22). The bias is bounded by the following equation:

$$\|b_r\|^2 \leq \Lambda(\delta_T, N) \cdot G_f^2. \quad (28)$$

$$\begin{aligned} \frac{1}{R} \sum_{r=0}^{R-1} \mathbb{E} \|\nabla f(W_r)\|^2 &\leq \underbrace{\frac{4[f(W_0) - f^*]}{\eta ER}}_{\text{(I)optimizationgap}} + \underbrace{4\eta L \Gamma \sigma^2}_{\text{(II)stochasticnoise}} + \underbrace{8\eta^2 L^2 E^2 \zeta^2}_{\text{(III)clientdrift}} + \underbrace{8\eta^2 L^2 E^2 G_f^2 \Lambda(\delta_T, N)}_{\text{(IV)EMAweightdiscrepancy}} \end{aligned} \quad (25)$$

The gradient deviation at subsequent steps $e \geq 1$ is bounded via the drift estimate in Lemma 2.

Step 3 (Variance Control): Decomposing $\mathbb{E}\|\bar{d}_r\|^2$ into signal and noise components, the noise term is bounded using Assumption 2 together with the independence of per-client sampling. The variance satisfies the following equation:

$$\mathbb{E}\|\bar{d}_r - \mathbb{E}[\bar{d}_r]\|^2 \leq \frac{\Gamma\sigma^2}{E} \quad (29)$$

where $\Gamma = \sum_u \gamma_u^2 \leq 1$ is the weight concentration factor.

Step 4 (Telescoping Sum): We now make the telescoping argument explicit. First, We substitute the bias bound [see (28)] and the variance bound [see (29)] into the per-round descent inequality (26). We then apply the inner-product relaxation $\langle a, b \rangle \geq 1/2\|a\|^2 - 1/2\|b - a\|^2$ with $a = \nabla f(W_r)$ and $b = \mathbb{E}[\bar{d}_r]$, which lower bounds the descent term and isolates the gradient norm $\|\nabla f(W_r)\|^2$. Together with the heterogeneity bound ζ^2 from Assumption 3 and the drift bound from Lemma 2, the single-round inequality becomes

$$\begin{aligned} \mathbb{E}f(W_{r+1}) &\leq \mathbb{E}f(W_r) - \frac{\eta E}{4}\mathbb{E}\|\nabla f(W_r)\|^2 + \eta^2 L E \Gamma \sigma^2 \\ &\quad + 2\eta^3 L^2 E^3 \zeta^2 + 2\eta^3 L^2 E^3 G_f^2 \Lambda(\delta_T, N). \end{aligned} \quad (30)$$

Rearranging the above inequality gives

$$\begin{aligned} \frac{\eta E}{4}\mathbb{E}\|\nabla f(W_r)\|^2 &\leq \mathbb{E}f(W_r) - \mathbb{E}f(W_{r+1}) + \eta^2 L E \Gamma \sigma^2 \\ &\quad + 2\eta^3 L^2 E^3 \zeta^2 \\ &\quad + 2\eta^3 L^2 E^3 G_f^2 \Lambda(\delta_T, N). \end{aligned} \quad (31)$$

Summing this inequality from $r = 0$ to $R - 1$ yields

$$\begin{aligned} \frac{\eta E}{4} \sum_{r=0}^{R-1} \mathbb{E}\|\nabla f(W_r)\|^2 &\leq f(W_0) - \mathbb{E}f(W_R) \\ &\quad + R\eta^2 L E \Gamma \sigma^2 + 2R\eta^3 L^2 E^3 \zeta^2 \\ &\quad + 2R\eta^3 L^2 E^3 G_f^2 \Lambda(\delta_T, N). \end{aligned} \quad (32)$$

Since $f(W_R) \geq f^*$, we have

$$\begin{aligned} \frac{1}{R} \sum_{r=0}^{R-1} \mathbb{E}\|\nabla f(W_r)\|^2 &\leq \frac{4[f(W_0) - f^*]}{\eta E R} \\ &\quad + 4\eta L \Gamma \sigma^2 + 8\eta^2 L^2 E^2 \zeta^2 \\ &\quad + 8\eta^2 L^2 E^2 G_f^2 \Lambda(\delta_T, N). \end{aligned}$$

This is the stated bound in (25), in which term (I) originates from the telescoped optimization gap, term (II) from the stochastic-noise variance, term (III) from the heterogeneity bound ζ^2 , and term (IV) from the EMA-induced weight discrepancy $\Lambda(\delta_T, N)$. If we set

$$\eta = \frac{c}{E\sqrt{R}} \quad (33)$$

where c is a positive constant satisfying the stability condition, terms (I) and (II) converge at rate $O(1/\sqrt{R})$, while the client drift terms (III) and the EMA discrepancy (IV) decay at rate $O(1/R)$. Therefore, the dominant convergence rate is determined by the first two terms, giving the following equation:

$$\frac{1}{R} \sum_{r=0}^{R-1} \mathbb{E}\|\nabla f(W_r)\|^2 = O\left(\frac{1}{\sqrt{R}}\right). \quad (34)$$

This completes the proof. $\blacksquare$

Corollary 1 (Role of the EMA Coefficient): Term (IV) in (25) is explicitly governed by δ_T . At $\delta_T = 1$, $\Lambda = 0$ and the bound reduces to the known FedAvg nonconvex result. For $\delta_T < 1$, the additional error is of order $O(\eta^2)$ and does not alter the dominant $O(1/\sqrt{R})$ rate, confirming that δ_T provides a continuously tunable tradeoff between global convergence and local personalization.

Corollary 2 (Bound-Guided Parameter Selection): The convergence bound in (25) provides a practical rule for selecting the main training parameters of *FSLog*. It links the learning rate η , the local epoch number E , the server-side EMA coefficient δ_T , and the communication round number R to the four error terms in the nonconvex bound. In (25), term (I) is the optimization gap, term (II) is the stochastic noise term, term (III) is the client drift term, and term (IV) is the EMA weight discrepancy term.

Learning Rate: Term (I) decreases when η increases, while term (II) increases with η . Balancing these two terms over R communication rounds and E local steps gives the theoretical scale $\eta = \Theta(1/E\sqrt{R})$. Under this scale, terms (I) and (II) share the dominant convergence rate $O(1/\sqrt{R})$. In practical deployment, the learning rate should also satisfy the stability condition $\eta \leq 1/4LE$. This explains the small fixed learning rate ($\eta = 0.0001$) adopted in our experiments, since a larger value inflates term (II) and prevents convergence, whereas a much smaller value enlarges term (I) and slows descent.

Local Epochs: The local epoch number E has two opposite effects. A larger E reduces term (I) through the factor $1/(\eta E R)$. However, it also enlarges terms (III) and (IV) through the factor E^2 . Therefore, an excessively large E amplifies client drift and EMA-induced discrepancy. A moderate value of E should balance local computation and drift accumulation. This agrees with the parameter sensitivity results in Section IV-D, where the F1 and M-F1 curves become stable at ($E = 10$).

Server-Side EMA Coefficient: The coefficient δ_T controls term (IV) through $\Lambda(\delta_T, N)$. From Definition 1, the discrepancy is bounded by $\Lambda(\delta_T, N) \leq 2(1 - \delta_T^N)^2$. Since $\Lambda(\delta_T, N)$ is monotonically decreasing in δ_T with $\Lambda(1, N) = 0$, a larger δ_T shrinks term IV toward the FedAvg limit, while a smaller δ_T strengthens personalization at the cost of an $O(\eta^2)$ residual. Under strong non-IID conditions, where ζ^2 in term (III) is large, a moderately large δ_T with 0.8 is preferred so that the personalization gain is retained without enlarging the dominant error. Overall, (25) provides a principled basis for the final parameter configuration used in Section IV-D and removes the previous reliance on purely empirical tuning.

Remark 1 (Role of the Client-Side EMA): The client H-layer EMA in (12) operates as a temporal exponential-decay smoother independent of the server. The initial bias decays at rate δ^t , while historical updates are weighted by $(1 - \delta)\delta^{r-1-t}$. Under the learning rate constraint of Theorem 1, this operation stabilizes the personalized head trajectory, complementing global convergence of the Transformer backbone.

Remark 2 (Comparison With Proximal-Regularization-Based PFL Methods): A representative alternative for mitigating client drift in federated nonconvex optimization is FedProx [55], which appends a proximal penalty $\mu/2\|W - W_{\text{global}}\|^2$

to each client's local objective. Under Assumptions 1–3, the FedProx convergence bound takes the form

$$\frac{1}{R} \sum_{r=0}^{R-1} \mathbb{E} \|\nabla f(W_r)\|^2 \leq O\left(\frac{1}{\sqrt{R}}\right) + O\left(\frac{\mu \zeta^2}{(\mu + L)^2}\right). \quad (35)$$

The second term constitutes an irreducible proximal bias that couples the regularization strength μ with the gradient dissimilarity ζ^2 . Increasing μ suppresses local drift but simultaneously inflates this bias floor, creating a rigid and nonvanishing tradeoff between convergence tightness and personalization capacity. By contrast, term (IV) in (25) of FL-EMA is governed by $\Lambda(\delta_T, N)$, which satisfies $\Lambda(1, N) = 0$ and is uniformly bounded by $2(1 - \delta_T^N)^2$.

As established in Corollary 1, this term is strictly $O(\eta^2)$ and does not alter the dominant $O(1/\sqrt{R})$ rate. More importantly, FL-EMA's overhead vanishes entirely at $\delta_T = 1$, gracefully recovering the standard FedAvg guarantee without residual bias. This is a degradation property that FedProx inherently lacks due to the persistent proximal penalty in its local objective. Client drift varies significantly in IoT syslog diagnosis under non-IID data distributions. FL-EMA overcomes this challenge via its structurally flexible design. By adaptively distributing personalization capacity, it completely avoids the rigid constraints of uniform proximal methods. Ultimately, this mechanism ensures highly effective local adaptation without sacrificing global generalization.

E. Fault Diagnosis

Due to differing granularity and latency requirements, we adopt a two-step approach: lightweight detection first identifies faults quickly, and then RCA precisely locates causes to enable effective repair, ensuring timeliness and accuracy. Fault diagnosis [47] aims to identify failure root causes and is often cast as classification; hence, we treat fault detection as a binary classification and RCA as a multiclass classification.

1) *Cloud Computing Network Offline Federated Training*: The VS aggregates protected intermediate representations from CS u . To ensure that E_θ is optimized consistently with the downstream diagnosis objective, its internal parameters θ are frozen. During forward propagation, the CS computes $A_u^{(i)}$, obtains $Z_u^{(i)}$, and uploads only $Z_u^{(i)}$ to the VS.

The VS performs Transformer computation and returns the cut-layer gradient during backpropagation. Using the chain rule, the CS back-propagates this gradient through the split boundary to update θ jointly with the H-layer via SGD. It performs log grouping to merge feature vectors and generate pseudo-labels for unlabeled samples.

It then trains a multiclass fault classifier via a bidirectional looped semi-supervised scheme, combining a supervised cross-entropy loss on few labeled samples with a weighted pseudo-supervision loss. Specifically, to mitigate the class imbalance issue, the weighted cross-entropy loss assigns higher penalty weights to minority classes during the training process. The weight w_c for each fault category c is inversely

proportional to its sample count N_c . The weighted supervised loss $\mathcal{L}_{sup}$ is formulated as

$$\mathcal{L}_{sup} = -\frac{1}{B} \sum_{b=1}^B \sum_{c=1}^C w_c y_{b,c} \log(p_{b,c} + \epsilon), w_c = \frac{1/N_c}{\sum_{j=1}^C 1/N_j} \quad (36)$$

where C is the total number of fault categories, y_c is the binary indicator of the true class label, and p_c is the predicted probability for class c . The penalty weight is normalized as w_c . This mechanism forces the model to focus more on the classification accuracy of minority classes, preventing majority classes from dominating the gradient updates. The result is a global multiclass RCA model. The VS distributes backbone parameters and pseudo-label knowledge to support client personalization.

2) *IoT Devices Network Offline Fine-tuning and Online Diagnosis*: During offline fine-tuning, for simple faults, the client uses a lightweight detector (e.g., a Transformer) for real-time binary threshold detection, thereby generating rapid fault alerts and caching fault samples. However, for complex faults, the CS receives the VS-distributed Transformer backbone and fine-tunes only the local head for offline personalization. During online detection, the fine-tuned local model computes softmax over linear outputs for feature $\phi_u(x^{(i)})$, and the class with maximum probability is taken as the multiclass fault label to identify fault type and RCA. For client u , the input $x^{(i)}$ passes through the personalized model $\mathcal{M}_u$ to obtain the feature representation $\phi_u(x^{(i)})$. The linear output for the m th fault class is given by the following equation:

$$o_m^u = (\omega_m^u)^T \phi_u(x^{(i)}) + \beta_m^u, \quad m = 1, 2, \dots, C \quad (37)$$

where $\phi_u(x^{(i)})$ is the feature representation from client u 's personalized model, ω_m^u and β_m^u are the weight and bias for the m th class in the local head, and C is the total number of fault categories. The probability for class m and the predicted label are given by the following equation:

$$p_m^u = \frac{\exp(o_m^u)}{\sum_{c=1}^C \exp(o_c^u)}, \quad \hat{y}_u = \arg \max_m p_m^u. \quad (38)$$

To clearly present CS and VS offline collaboration of *FSLog*, we provide Algorithms 2 and 3.

IV. EVALUATION

In this section, we evaluate the performance of *FSLog* by addressing the following research questions (RQs).

- 1) *RQ1*: How does *FSLog* performance compare to baseline methods in the fault diagnosis of large-scale systems?
- 2) *RQ2*: Do the core techniques of FL-EMA and SL-Bert significantly contribute to the performance of *FSLog*?
- 3) *RQ3*: How do the major parameters of *FSLog* influence its performance on the RCA task?

A. Experimental Design

1) *Datasets*: The experiments were conducted on two datasets collected from IoT networks, as shown in Table II.

Algorithm 2 CS Procedure of FL-EMA With Split Bert

Require: Local epoch E , batch size B , training round R , learning rate η , client-side Head EMA coefficient δ_H^u .
Ensure: Personalized local model $\mathcal{M}_u$

- 1 Initialize frozen client-server E-layer W_E^u , irreversible activation encoder E_θ , H-layer $W_{H^0}^u$, and smoothed Head $\bar{W}_{H^0}^u \leftarrow W_{H^0}^u$.
- 2 $\mathcal{B}_u \leftarrow \text{MiniBatch}(\tilde{D}_{\text{train}}^u, B)$
- 3 Freeze W_E^u during FL-EMA training; only the local H-layer W_H^u is updated on the client-server.
- 4 **for** each round $r = 0, 1, \dots, R-1$ **do**
- 5 **for** each local epoch $e = 1, 2, \dots, E$ **do**
- 6 **for** batch $(x_b, y_b) \in \mathcal{B}_u$ **do**
- 7 $\text{embTensor}_u \leftarrow W_E^u(x_b)$
- 8 $\text{embTensor}_u \leftarrow \text{Filter}_\tau(\text{embTensor}_u)$
- 9 $Z_u \leftarrow E_\theta(\text{embTensor}_u)$ Eq.(5)
- 10 send Z_u to vendor-server
- 11 wait to receive tfTensor_u from vendor-server
- 12 $\hat{y}_u \leftarrow H_{W_H^u}(\text{tfTensor}_u)$
- 13 $\mathcal{L}_u \leftarrow \ell(\hat{y}_u, y_b)$
- 14 compute $\nabla_{W_H^u} \mathcal{L}_u$ and $G_{\text{tf}}^u \leftarrow \frac{\partial \mathcal{L}_u}{\partial \text{tfTensor}_u}$
- 15 $W_{H^r}^u \leftarrow W_{H^{r-1}}^u - \eta \cdot \nabla_{W_H^u} \mathcal{L}_u$
- 16 send G_{tf}^u to vendor-server
- 17 **end for**
- 18 **end for**
- 19 $W_{H^r}^{u*} \leftarrow W_{H^r}^u$
- 20 **// Local Head EMA**
- 21 $\bar{W}_{H^{r+1}}^{\text{ema}(u)} \leftarrow \delta_H^u \bar{W}_{H^r}^{\text{ema}(u)} + (1 - \delta_H^u) W_{H^r}^{u*}$ Eq.(12)
- 22 $W_{H^{r+1}}^u \leftarrow \bar{W}_{H^{r+1}}^{\text{ema}(u)}$
- 23 **end for**
- 24 wait to receive $W_{T^R}^{\text{ema}(s)}$ from vendor-server
- 25 $\Theta_u \leftarrow \{W_E^u, E_\theta, W_{T^R}^{\text{ema}(s)}, \bar{W}_{H^R}^{\text{ema}(u)}\}$ Eq.(13)
- 26 $\mathcal{M}_u(x; \Theta_u) \leftarrow H_{\bar{W}_{H^R}^{\text{ema}(u)}}(T_{W_{T^R}^{\text{ema}(s)}}(E_\theta(\text{Filter}_\tau(W_E^u(x))))$ Eq.(14)
- 27 **return:** $\mathcal{M}_u$

TABLE II
DETAILS OF THE DATASETS

Datasets	# of logs	# of fault categories	# of log sequences	# of fault log sequences
Aliyun	482,515	3	35,960	28,023
CMCC	299,817	6	29,662	18,792

a) *Aliyun*¹: It is an operational syslog sourced from the Alibaba Cloud Tianchi platform from production servers, which comprises 13 700+ server numbers across 87 server models. It contains 482 515 log entries recorded over six weeks. After log parsing via Drain and sliding time window segmentation, these logs yield 35 960 log sequences, of which 28 023 are fault log sequences. Each fault log sequence is mapped to one of three root causes: processor internal faults, memory conflicts, and peripheral hardware failures.

b) *China Mobile Communications Corporation*: The dataset of VSwitch originates from China Mobile Communications Corporation (CMCC) OpenStack-based system [69].

Algorithm 3 VS Procedure of FL-EMA With Split Bert

Require: Local epoch E , training round R , number of CSs N , client data volumes $\{|D_u|\}_{u=1}^N$ with $|D| = \sum_{u=1}^N |D_u|$, learning rate η , server-side Transformer EMA coefficient δ_T .
Ensure: EMA-smoothed Transformer backbone $W_{T^R}^{\text{ema}(s)}$.

- 1 Initialize the vendor-server Transformer backbone $W_{T^0}^{\text{ema}(s)}$
- 2 $S_r \leftarrow \{1, 2, \dots, N\}$ for every round r
 // full participation
- 3 **for** each round $r = 0, 1, \dots, R-1$ **do**
- 4 **for** each client-server $u \in S_r$ **in parallel do**
- 5 $W_{T^r}^u \leftarrow W_{T^r}^{\text{ema}(s)}$
- 6 **for** each local epoch $e = 1, 2, \dots, E$ **do**
- 7 **while** a protected representation Z_u is received from client-server u **do**
- 8 $\text{tfTensor}_u \leftarrow T_{W_{T^r}^u}(Z_u)$
- 9 send tfTensor_u to client-server u
- 10 wait to receive $G_{\text{tf}}^u = \frac{\partial \mathcal{L}_u}{\partial \text{tfTensor}_u}$ from client-server u
- 11 $\nabla_{W_{T^r}^u} \mathcal{L}_u \leftarrow \text{Backprop}_T(G_{\text{tf}}^u, Z_u; W_{T^r}^u)$
- 12 $W_{T^r}^u \leftarrow W_{T^r}^u - \eta \nabla_{W_{T^r}^u} \mathcal{L}_u$
- 13 **end while**
- 14 **end for**
- 15 $W_{T^r}^{u*} \leftarrow W_{T^r}^u$
- 16 **end for**
- 17 **// FedAvg: aggregate Transformer only**
- 18 $W_{T^{r+1}}^{\text{agg}} \leftarrow \sum_{j=1}^N \frac{|D_{u_j}|}{|D|} W_{T^r}^{u_j}$ Eq.(8)
- 19 **// Sequential Transformer EMA over the ordered clients $u_1, \dots, u_N$**
- 20 $W_{T^{r+1}}^{\text{ema}(s_0)} \leftarrow W_{T^{r+1}}^{\text{agg}}$ Eq.(9)
- 21 **for** $j = 1, 2, \dots, N$ **do**
- 22 $W_{T^{r+1}}^{\text{ema}(s_j)} \leftarrow \delta_T W_{T^{r+1}}^{\text{ema}(s_{j-1})} + (1 - \delta_T) W_{T^r}^{u_j}$ Eq.(10)
- 23 **end for**
- 24 $W_{T^{r+1}}^{\text{ema}(s)} \leftarrow W_{T^{r+1}}^{\text{ema}(s_N)}$ Eq.(11)
- 25 **end for**
- 26 **for** each client-server $u = 1, 2, \dots, N$ **in sequence do**
- 27 send $W_{T^R}^{\text{ema}(s)}$ to client-server u
- 28 **end for**
- 29 **return:** $W_{T^R}^{\text{ema}(s)}$

It was collected from a high-performance computing cluster, with 493 nodes over one month. The raw dataset comprises 299 817 log entries. After log parsing and sliding time window segmentation, these logs are transformed into 29 662 log sequences, among which 18 792 are fault log sequences. The fault log sequences are categorized into one of six root causes: software bug, operational port down, management port down, virtual machine port down, virtual machine start-up fault, and misconfiguration.

2) *Experimental Setup*: All experiments are conducted within a Docker environment. The host machine features dual Intel Xeon Gold 5218 CPUs (at 2.3 GHz), one NVIDIA Tesla V100S GPU, and 192-GB RAM. Deep learning training leverages GPU acceleration within Docker containers. The system *FSLog* is implemented with Python 3.7.1, PyTorch 1.9.0, and CUDA 10.2. On the datasets, 80% of fault log

¹<https://tianchi.aliyun.com/competition/entrance/531947/information>

sequences are used for training and 20% for testing. A Docker-based FL simulator is built on a swarm-orchestrated multicontainer architecture, which simulates real-world federated settings. Specifically, for both the Aliyun and CMCC dataset evaluations, the simulator explicitly instantiates five CS Dockers and one VS Docker. Each CS hosts a local model that encompasses all the local nodes within its corresponding partition, whereas the VS maintains the globally aggregated model. This deployment topology realizes the collaborative training between five personalized local models and one global model. For the Aliyun dataset, the 87 server models are partitioned among the five CSs by business-affiliation clustering, yielding {18, 17, 18, 17, 17} server models per CS. For the CMCC dataset, the 493 client nodes are partitioned among the five CSs under a tenant-isolation scheme, yielding {99, 99, 98, 99, 98} nodes per CS. All six Dockers are orchestrated by Docker Swarm on a single host and interconnected through a dedicated overlay network.

3) *Evaluation Metrics*: The performance of fault diagnosis is evaluated based on two stages: fault detection and RCA.

a) *Diagnostic utility*: We utilize the F1-score (F1) for binary fault detection and the macro-averaged F1-score (M-F1) for the multiclass RCA task. Fault detection involves analyzing syslog sequences to identify anomalies. To calculate this, we first define *Precision* (P) and *Recall* (R). *Precision* measures the proportion of true positives (TPs) among all predicted positives, while *Recall* measures the proportion of TPs among all actual positives. These are defined as defined in the following equations:

$$Precision = \frac{TP}{TP + FP} \quad (39)$$

$$Recall = \frac{TP}{TP + FN} \quad (40)$$

where *FP* and *FN* denote false positives and false negatives, respectively. The F1-score is then derived as the harmonic mean of Precision and Recall, balancing the tradeoff between the two metrics. For the i th fault category, its F1-score ($F1_i$) is calculated as shown in the following equation:

$$F1_i = \frac{2 \cdot Precision_i \cdot Recall_i}{Precision_i + Recall_i} \quad (41)$$

For the multiclass RCA task, we utilize M-F1 to evaluate the comprehensive diagnostic performance. M-F1 is calculated by first obtaining the F1-score for each fault class and then taking the arithmetic mean of these class-wise F1-scores for each distinct fault class and subsequently computes the arithmetic mean of these individual scores. Consequently, it systematically mitigates assessment biases dominated by majority classes and provides an objective insight into the model's diagnostic capability across all fault types. The final formulation is defined in the following equation, where n represents the total number of fault categories:

$$M-F1 = \frac{1}{n} \sum_{i=1}^n F1_i \quad (42)$$

b) *Privacy evaluation metrics*: To quantify privacy leakage alongside diagnostic utility, we utilize the attack success

rate (ASR). Let a_i be the ground-truth sensitive attribute of sample i . The attack model A_ψ , parameterized by ψ , is implemented as a lightweight multilayer perceptron classifier. This classifier takes the mean-pooled transmitted representation $Z_u^{(i)}$ as input to predict the corresponding sensitive attribute. Consequently, the ASR is formally defined as

$$ASR = \frac{1}{M} \sum_{i=1}^M \mathbb{I}[A_\psi(Z_u^{(i)}) = a_i] \quad (43)$$

A lower ASR value indicates a stronger empirical resistance against attribute inference. We report the ASR alongside the downstream M-F1 to comprehensively characterize the privacy-utility tradeoff.

B. RQ1: Overall Performance of FSLog

For a comprehensive assessment of *FSLog* in fault diagnosis, we compare it with ten baselines covering seven fault detection methods and three RCA methods. In order to highlight the performance gap between centralized and collaborative paradigms, the fault detection baselines are organized into two groups: 1) traditional centralized methods, including DeepLog [36], LogAnomaly [35], LogRobust [37], and DistilLog [52]; and 2) distributed collaborative methods under federated settings, including AdaLightLog [51], FedLog [11], and FLUXLog [56]. In addition, three methods capable of multiclass RCA evaluation are adopted: NeuralLog [41], Cloud19 [45], and LogCluster [44]. All the baselines need to be federated in advance because DV can only capture raw syslog data with unified business scenarios in mind. Since the seven fault detection baselines are designed for binary anomaly classification and do not support multiclass categorization, their M-F1 are not applicable and are denoted as “-.”

As shown in Table III, *FSLog* consistently outperforms all baselines on tasks of fault detection and RCA in IoT. On Aliyun, *FSLog*'s M-F1 and F1 are 0.950 and 0.952, exceeding the strongest Cloud19 by 10.6% and 2.2%, and on the CMCC, M-F1 and F1 are 0.960 and 0.964, surpassing the strongest NeuralLog by 6.0% and 2.8%.

1) *Traditional Centralized Fault Detection*: Among the centralized baselines, LogAnomaly achieves the highest F1 of 0.890 on Aliyun through its semantic vector encoding with Bi-LSTM, while DistilLog obtains the highest F1 of 0.916 on CMCC, with a balanced precision and recall tradeoff that validates the effectiveness of knowledge distillation for compact model deployment. However, these centralized methods generally yield lower F1 scores than FLUXLog under the FL framework on Aliyun.

2) *Distributed Collaborative Fault Detection*: Among the federated baselines, FLUXLog achieves the highest F1 of 0.892 on Aliyun through its MoE routing mechanism. However, its F1 drops to 0.832 on CMCC, revealing limited stability under strong non-IID client distributions where expert routing lacks sufficient training samples, thereby increasing misclassification. FedLog obtains F1 values of 0.886 and 0.900 on the two datasets, benefiting from its native federated architecture. AdaLightLog yields the lowest F1 among the federated baselines, as its structural adaptation mechanism

TABLE III
PERFORMANCE OF FSLog AND BASELINE METHODS

Method	Aliyun				CMCC			
	M-F1	F1	P	R	M-F1	F1	P	R
AdaLightLog	-	0.843	0.920	0.778	-	0.826	1.000	0.704
DistillLog	-	0.869	0.844	0.896	-	0.916	0.876	0.959
DeepLog	-	0.866	0.764	1.000	-	0.850	0.739	1.000
FedLog	-	0.886	0.796	1.000	-	0.900	0.819	1.000
FLUXLog	-	0.892	0.916	0.870	-	0.832	0.713	1.000
LogAnomaly	-	0.890	0.802	1.000	-	0.848	0.736	1.000
LogRobust	-	0.860	0.755	1.000	-	0.766	0.621	1.000
Cloud19	0.844	0.930	0.963	0.900	0.704	0.844	0.904	0.792
NeuralLog	0.646	0.890	0.996	0.805	0.900	0.936	0.916	0.956
LogCluster	0.786	0.920	0.880	0.964	0.666	0.828	0.813	0.844
FSLog	0.950	0.952	0.950	0.954	0.960	0.964	0.951	0.978

prioritizes local heterogeneity accommodation at the cost of global detection consistency.

3) *Root-Cause Analysis*: On Aliyun, Cloud19 demonstrates the strongest baseline RCA performance besides our proposed *FSLog*. However, on the CMCC, Cloud19's M-F1 drops to 0.704, indicating limited effectiveness in fault categorization across diverse datasets. This suggests that when fault categories become more diverse and client distributions become more heterogeneous, its Word2Vec-based shallow embeddings lack sufficient robustness. In addition, NeuralLog shows strong performance on the CMCC, indicating that its integration of Transformer models effectively captures contextual information, enabling a better understanding of complex log semantics. However, NeuralLog's M-F1 on Aliyun is only 0.646, despite its Precision reaching 0.996. This discrepancy reveals that accurate binary fault discrimination does not necessarily translate into reliable multiclass root cause identification. LogCluster obtains moderate M-F1 of 0.786 and 0.666 on the two datasets. Its clustering-based representation offers computational efficiency but lacks the capacity to model deep semantic correlations and cross-client distribution shifts. These results demonstrate that *FSLog* is well suited for IoT deployments that require both timely fault detection and accurate RCA.

C. RQ2: Ablation Study

We perform ablation experiments on the two datasets to evaluate the effectiveness of two modules of *FSLog*, i.e., FL-EMA and SL-Bert.

1) Effectiveness of FL-EMA Personalized Local Models:

To systematically evaluate the contribution of each component in the FL-EMA mechanism, we conduct three groups of ablation experiments: removing the entire FL-EMA module (w/o FL-EMA), removing only EMA while retaining FedAvg (w/o EMA), and removing only FedAvg while retaining EMA (w/o FedAvg). The results are reported in Table IV.

a) *Removing FL-EMA*: When the entire FL-EMA module is removed, both global and local models suffer severe performance degradation on both datasets. On Aliyun, the global model M-F1 drops sharply from 0.950 to 0.300, and all local models exhibit substantial M-F1 declines, with most falling below 0.500. On the CMCC, the global model M-F1

decreases from 0.960 to 0.220, while the average local model M-F1 falls below 0.260. This collapse demonstrates that the FL-EMA module is indispensable for maintaining diagnostic accuracy under heterogeneous data distributions.

b) *Removing ema*: We remove EMA from *FSLog*, and the results show that the performance of *FSLog* without EMA is degraded. On Aliyun, the F1 and M-F1 are 0.912 and 0.838, respectively, which are 4% and 11.2% lower than the *FSLog* performance with EMA. Moreover, certain local models also exhibit notable degradation. For instance, Local model ② on the CMCC drops from 0.926 to 0.812 in M-F1, and Local Model ③ drops from 0.906 to 0.652. The degradation after removing EMA is not uniform across clients. It is governed by three client-level characteristics. First, the intraclient non-IID severity matters most. A client whose partition aggregates semantically divergent server models and skewed fault-category distributions induces a larger gradient-dissimilarity ζ^2 , which corresponds to term (III) of the convergence bound in (25). Second, the local sample volume matters. A client with a smaller data proportion p_u is more exposed to the stochastic-noise variance σ^2/E in term (II). Third, the boundary stability matters. A noisier local classification boundary benefits more from the temporal smoothing in (12). These three factors jointly explain the observed pattern. Local Model ① and Local Model ⑤ on Aliyun, together with Local Model ③ on CMCC, host the most heterogeneous and most imbalanced partitions, so they suffer the largest M-F1 drops once temporal smoothing is removed. By contrast, Local Model ④ on Aliyun hosts a comparatively homogeneous and near-balanced partition, so its local trajectory is already stable, and its degradation stays minimal. This correspondence between partition heterogeneity and EMA sensitivity is consistent with Theorem 1. It confirms that EMA contributes adaptively, in the sense that the clients with the strongest drift gain the most stabilization while the already-stable clients are left essentially unaffected.

c) *Removing FedAvg*: When FedAvg is removed, the global model performance degrades drastically. On Aliyun, the global model M-F1 decreases from 0.950 to 0.620, and on the CMCC, it drops from 0.960 to 0.260. Although certain local models retain relatively high performance (e.g., Local Model ③ achieves an M-F1 of 0.900), the overall cross-client consistency deteriorates significantly. This observation confirms that FedAvg is essential for establishing a strong global consensus, upon which the subsequent EMA personalization depends.

From Table IV, we observe that the complete FL-EMA mechanism, combining forward FedAvg aggregation with backward EMA personalization, consistently achieves the highest and most stable performance across both global and local models. Neither FedAvg nor EMA alone can match the performance of their synergistic combination. Therefore, we conclude that the EMA of *FSLog* is better adapted to the personalized local models under different devices. We can get the global model after using FedAvg when the communication intersection is backpropagated to the T-layer. Following this, EMA is applied to update the local model in a way that the whole fine-tuning process is a multiround process of aggregat-

TABLE IV
PERFORMANCE COMPARISON OF GLOBAL MODEL AND LOCAL MODELS UNDER COMPONENT ABLATION

Model	Aliyun								CMCC							
	<i>FSLog</i>		w/o FL-EMA		w/o EMA		w/o FedAvg		<i>FSLog</i>		w/o FL-EMA		w/o EMA		w/o FedAvg	
	M-F1	F1	M-F1	F1	M-F1	F1	M-F1	F1	M-F1	F1	M-F1	F1	M-F1	F1	M-F1	F1
Global Model	0.950	0.952	0.300	0.810	0.838	0.912	0.620	0.870	0.960	0.964	0.220	0.640	0.682	0.918	0.260	0.640
Local Model ①	0.930	0.970	0.410	0.880	0.730	0.970	0.630	0.880	0.978	0.946	0.260	0.760	0.936	0.926	0.920	0.920
Local Model ②	0.980	0.920	0.750	0.780	0.920	0.720	0.910	0.920	0.926	0.946	0.170	0.630	0.812	0.934	0.620	0.910
Local Model ③	0.920	0.980	0.402	0.890	0.810	0.960	0.900	0.890	0.906	0.966	0.180	0.880	0.652	0.960	0.840	0.940
Local Model ④	0.940	0.970	0.310	0.900	0.930	0.970	0.710	0.900	0.954	0.950	0.410	0.810	0.910	0.944	0.560	0.900
Local Model ⑤	0.980	0.960	0.780	0.850	0.800	0.940	0.880	0.970	0.984	0.940	0.230	0.890	0.902	0.922	0.250	0.930



Fig. 3. FLOPs under different split points k on the two datasets. (a) Aliyun and (b) CMCC.

ing models on the VS. The experimental results in Table IV show that personalized local models under the fine-tuning of this strategy are more adaptive. The experimental observations are corroborated by the convergence analysis in Section III-D3. These observations are consistent with the theoretical analysis: removing EMA weakens temporal smoothing and makes local trajectories more unstable under non-IID distributions.

2) *Effectiveness of Split Bert With Computation Load Balancing and Privacy Protection*: To validate the effectiveness of the split Bert model with privacy-preserving chains, we conduct experiments comparing two aspects: client computation load balancing and privacy protection.

a) *Computational Load*: To validate the load balancing effectiveness of SL-Bert, we evaluate the FLOPs distribution under different split positions. Since the parameter sensitivity study in Section IV-D has fixed the number of Transformer encoder layers at $l = 3$, we define k as the number of Transformer layers retained on the CS, so that $l - k$ layers are offloaded to the VS, with the split point constrained by $k \leq l$, namely, $k \in \{0, 1, 2, 3\}$. On this basis, we compare the following baselines: 1) w/o SL-Bert, that is $k = 3$, where the complete Bert model is deployed locally; and 2) SL-Bert under different split points, that is $k \in \{0, 1, 2\}$, where $k = 1$ and $k = 2$ are intermediate splits that retain one or two layers locally, and $k = 0$ denotes *FSLog*, which corresponds to the E/T boundary split adopted, where no Transformer layer remains on the CS and the entire backbone is offloaded to the VS.

i) *Effect of the split point (k)*: Fig. 3 reports the CS and VS FLOPs under different values of k . When SL-Bert is removed, the complete Bert model is deployed on the CS, so both forward and backward propagation are executed locally. Under this configuration, the VS only handles lightweight operations such as parameter reception and training control, and its FLOPs are therefore approximately zero. As a

result, all training bottlenecks concentrate on the resource-constrained CS, which readily causes computational overload and communication latency on edge devices. After SL-Bert is introduced, the architecture follows an edge-cloud partition that migrates the training workload from the CS to the VS. We split the backbone at different layers with $k \in \{0, 1, 2, 3\}$ on both datasets. The results show that a larger k moves the split point deeper into the backbone, so the CS FLOPs grow approximately linearly, since each retained layer reintroduces one round of self-attention and feed-forward computation. Specifically, on the Aliyun dataset, the average CS FLOPs rise from 7G at $k = 0$ to 2332G, 4658G, and 6983G at $k = 1$, $k = 2$, and $k = 3$, while the average VS FLOPs decrease correspondingly from 6976G to 4651G, 2325G, and finally to nearly 0G at the no-split reference. The CMCC dataset exhibits the same trend, where the average CS FLOPs rise from 4G to 3953G across the four split points. Hence, SL-Bert reduces the client-side computational load by about 99.8% in terms of FLOPs. It should be noted that this reduction is a FLOPs-based estimate. Our evaluation is conducted in a Docker-based simulation on server-class hardware, and the two datasets consist of datacenter-server syslogs collected through gateway-class CSs. Accordingly, the CS in our deployment corresponds to a gateway-class edge server that aggregates logs from its managed end devices, rather than a deeply constrained end node itself. Nevertheless, the boundary split $k = 0$ retains only the lightweight embedding, the bottleneck encoder, and the linear head on the CS, so the residual client-side load remains well within the capability of such gateway hardware.

b) *Privacy evaluation under adversarial threat model*: To comprehensively evaluate the privacy-preserving capability of SL-Bert, we further evaluate whether the selected split point can resist privacy leakage under attribute inference attacks. We conduct structured attribute inference attacks under the threat model defined below. The attack model A_ψ and the ASR follow the formal definitions given in Section IV-A and (43), and are not restated here.

i) *Threat model and adversarial capability*: We consider an honest-but-curious VS, following established privacy threat models [70]. The adversary is strictly passive. It complies with the training protocol and only performs attribute inference, without modifying the uploaded representation, perturbing the forward computation, or interfering with training. Concretely, the adversary observes the uploaded protected representation, the cut-layer gradients, the model architecture, and the training

TABLE V
ASR AND M-F1 UNDER DIFFERENT PRIVACY CHAINS AND SL-BERT SPLIT POINTS

Dataset	CE	Attack Scenario	w/o any privacy chain				M-F1	w/o irreversible activation		FSLog (full privacy chain)							
			ASR					ASR	M-F1	ASR				M-F1			
			k=0	k=1	k=2	k=3				k=0	k=1	k=2	k=3	k=0	k=1	k=2	k=3
Aliyun	CE ₁	Expose area	42%	35%	32%	30%	0.972	20%	0.842	18%	17%	17%	15%	0.950	0.952	0.955	0.956
	CE _Ω	Infer server IP	38%	33%	30%	28%		26%		22%	21%	20%	18%				
CMCC	CE ₁	Expose area	45%	39%	36%	35%	0.975	22%	0.855	20%	19%	18%	18%	0.960	0.964	0.968	0.970
	CE _Ω	Infer server IP	40%	35%	32%	30%		28%		25%	24%	25%	22%				

protocol. It may further train a shadow inference classifier on auxiliary representations produced by the same preprocessing pipeline. However, it cannot access the raw syslogs, the local sensitivity dictionaries, the E-layer parameters, the irreversible encoder parameters, the personalized H-layer parameters, or the local labels retained on the CS. Accordingly, its goal is not to reconstruct the raw syslog, but to infer the sensitive attribute carried by the uploaded representation. We examine two representative targets, namely, Expose area and Infer server IP. For the former, the target is the deployment area or tenant region. For the latter, the target is the anonymized server identity or IP-prefix class rather than the full raw IP string. Because the adversary never alters the transmitted activation, the attack affects only privacy leakage and leaves the diagnostic accuracy unchanged. Consequently, the ASR and the M-F1 are governed by independent factors throughout this evaluation.

ii) *Evaluation Configurations*: To characterize how the split point interacts with each privacy layer, we vary the number of Transformer layers retained on the CS over $k \in \{0, 1, 2, 3\}$ and report the ASR together with the downstream M-F1 in Table V. The evaluation spans three configurations of increasing protection.

- 1) w/o any privacy chain (raw activation, which removes both feature filtering and irreversible activation).
- 2) w/o irreversible activation, which retains only the single feature filtering. Since it isolates the marginal effect of one privacy layer, which is reported at the default boundary split ($k = 0$) under the identical setting as the full chain.
- 3) *FSLog* with the full privacy chain, which combines feature filtering and the irreversible encoder E_θ over all split points.

Three observations follow. First, regarding privacy, the configuration without any privacy chain exhibits a clear coupling between split depth and leakage. As k increases, the CS retains more Transformer layers, so the uploaded activation becomes deeper, more contextualized, and harder to invert, and the ASR therefore decreases monotonically from 38% to 45% at $k = 0$ to 28%–35% at $k = 3$. This confirms that the split depth itself contributes to privacy. Under the full privacy chain, the same depth effect persists but is strongly attenuated, so the ASR declines only mildly and stays within a low band of 15%–25% across all split points. In other words, the privacy chain absorbs most of the attack surface and decouples the privacy guarantee from the split depth, whereas the residual depth effect remains because the chain

offers empirical rather than absolute resistance under the threat model.

Second, regarding utility, the split depth barely affects the M-F1. Under the unprotected reference, the split is lossless and does not change the representational capacity of the model, so the M-F1 remains constant at 0.972 and 0.975 on the two datasets. Under the full privacy chain, the M-F1 rises only slightly with k , from 0.950 to 0.956 on Aliyun and from 0.960 to 0.970 on CMCC, because the additional local layers contextualize the filtered embedding before the low-dimensional compression and thus preserve more task-relevant information. The gain saturates beyond $k = 2$, and the boundary split concedes at most 2 points of M-F1 relative to the deepest tested split.

Third, regarding the role of the irreversible encoder, removing it does not aggravate the utility loss but instead worsens it. With feature filtering alone, the M-F1 drops to 0.842 and 0.855 because aggressive token-level filtering removes part of the diagnosis-relevant semantics. Adding the irreversible encoder recovers the M-F1 to 0.950 and 0.960 while keeping the ASR in the lowest band. The encoder is trained end-to-end with the task loss, so it compensates for the semantics discarded by filtering and simultaneously reduces leakage. Therefore, the full chain attains a higher M-F1 than the filtering-only variant rather than a lower one, and the two layers jointly form an effective privacy–utility tradeoff.

Taking client efficiency, privacy protection, and diagnostic utility into account, the three dimensions point to a consistent operating point. A deeper split yields only a marginal privacy gain and a marginal utility gain, yet it raises the client-side FLOPs almost linearly and thus violates the resource-constrained nature of IoT clients. The boundary split $k = 0$ minimizes the client computation and reduces the client-side FLOPs by approximately 99.8%. Meanwhile, the two-layer privacy chain holds the ASR within 15%–25% at this point while preserving the M-F1, with a utility loss of only 2.2 percentage points. Hence, $k = 0$ is a deliberate utility-cost tradeoff and the preferred configuration of *FSLog*.

D. RQ3: Parameter Sensitivity

We investigate how the major hyperparameters of *FSLog* affect its performance on both the fault detection and the RCA tasks. Meanwhile, we examine seven hyperparameters in turn, namely, the local training epoch E , the protected latent dimension d , the number of Transformer encoder layers l , the learning rate η , the sliding window size w , and the EMA coefficients δ_T and δ_H^u . Guided by the convergence analysis in Corollary 2, the following discussion identifies the optimal

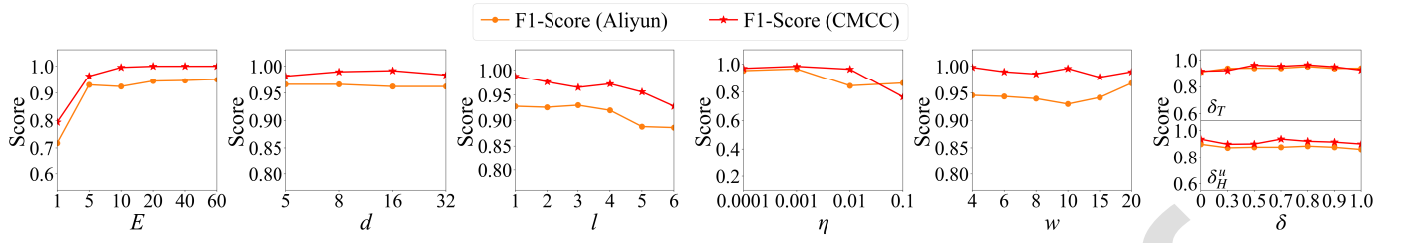


Fig. 4. F1-score for fault detection with different hyperparameters E , d , l , η , w , and δ on the two datasets.

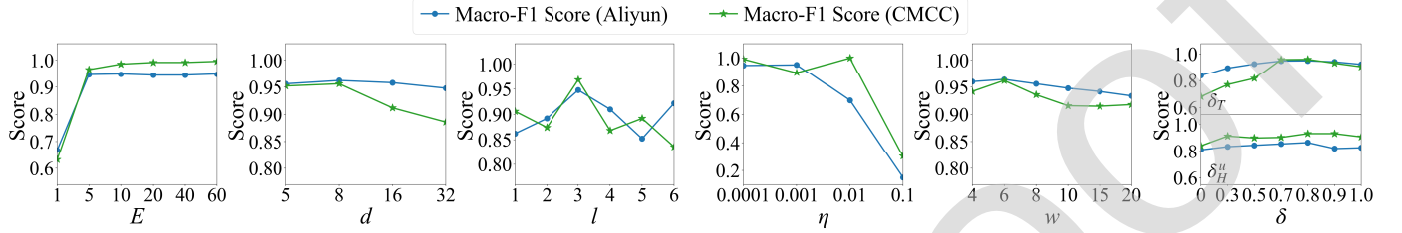


Fig. 5. Macro-F1 Score for RCA with different hyperparameters E , d , l , η , w , and δ on the two datasets.

value of each hyperparameter and explains the underlying trend.

1) *Local Training Epoch (E)*: The local training epoch E determines the number of local update steps performed on each client before aggregation, and it therefore governs the tradeoff between diagnostic accuracy and training cost. We vary E over $\{1, 5, 10, 20, 40, 60\}$ on both datasets, with the results reported in the first subfigures of Figs. 4 and 5. When $E = 1$, both the F1 and the M-F1 remain at a low level, since a single local step is insufficient for the client model to fit the syslog semantics. As E increases from 1 to 5, both metrics rise steeply, which indicates that additional local updates substantially improve the local representation. Beyond this point, the curves flatten and reach convergence at $E = 10$ on both datasets. When E further grows to 20, 40, and 60, the metrics exhibit no appreciable gain, whereas the training time and the computational cost continue to increase. This behavior is consistent with the convergence analysis in Corollary 2, where an excessively large E amplifies the client-drift and EMA-discrepancy terms through the factor E^2 and thus offers no further benefit. Therefore, by balancing diagnostic performance against training cost, we set the local training epoch to $E = 10$ as the optimal configuration.

2) *Protected Latent Dimension (d)*: The irreversible activation encoder E_θ maps the filtered E-layer representation into a protected, low-dimensional latent space d . Specifically, it compresses the feature dimension from the original embedding size d_e to d , while strictly preserving the log sequence length n . Thus, we study the effect of the protected latent dimension and present the results in the second subfigures of Figs. 4 and 5 with $d \in \{5, 8, 16, 32\}$. As observed, when $d = 5$, the protected representation is too compact to capture valuable diagnostic features. However, when d reaches 8, both the F1 score and M-F1 converge and achieve stability. Therefore, $d = 8$ is selected as the smallest stable protected dimension.

3) *Number of Transformer Encoder Layers (l)*: The number of encoder layers l governs the representational depth of SL-Bert and therefore affects both diagnostic accuracy and

server-side computation. We vary l from 1 to 6 under the split configuration $k = 0$, so that all encoder layers reside on the VS and the client FLOPs remain constant across all settings. The results are shown in the third subfigures of Figs. 4 and 5. On both datasets, the M-F1 first rises with l and then declines, which forms a clear single-peak pattern. The peak occurs at $l = 3$, where the Aliyun M-F1 reaches 0.950, and the CMCC M-F1 reaches 0.960. A shallow encoder with $l \leq 2$ underfits the syslog semantics and yields lower accuracy. In contrast, a deeper encoder with $l \geq 4$ does not bring further gains and even degrades performance, since the limited labeled fault sequences are insufficient to train the additional parameters and thus induce overfitting. Meanwhile, the server FLOPs grow linearly with l , so a larger l raises the computational cost on the VS without improving accuracy. Considering the joint tradeoff between diagnostic accuracy and computational cost, we set $l = 3$ as the default encoder depth.

4) *Learning Rate (η)*: The learning rate η controls the step size of the local stochastic gradient descent, and it therefore directly affects both the stability and the final accuracy of the model. We evaluate η over $\{0.0001, 0.001, 0.01, 0.1\}$ on both datasets, with the results reported in the fourth subfigures of Figs. 4 and 5. When $\eta = 0.1$, the model fails to converge on both datasets, since such a large step size causes the optimization to oscillate and diverge. As η decreases to 0.01 and then to 0.001, the accuracy improves steadily, which indicates that a smaller step size yields a more stable descent trajectory. The best performance appears at $\eta = 0.0001$, where the F1 and the M-F1 reach their highest values on both datasets. This trend is consistent with the convergence analysis in Corollary 2, in which a larger η inflates the stochastic-noise term and prevents convergence, whereas the chosen $\eta = 0.0001$ satisfies the stability condition $\eta \leq 1/(4LE)$. Although a smaller learning rate requires more iterations to converge, this cost is acceptable because it guarantees a stable optimization and the highest diagnostic accuracy. By jointly considering accuracy and convergence stability, we set $\eta = 0.0001$ as the optimal configuration.

5) *Sliding Window (w)*: The sliding window determines how many consecutive parsed log events are used to construct one diagnostic sequence. A small value captures only short symptoms. It may miss the follow-up state transition and confirmation messages that are important for RCA. A large value introduces more background logs into the same sequence. This may blur the boundary between different root causes. As shown in the fifth subfigures of Figs. 4 and 5, $w = 6$ provides the most stable joint performance on the two datasets. It maintains high F1-score for fault detection and achieves the best or near-best M-F1 for RCA. When w increases to 10, 15, or 20, the fault detection score remains acceptable in some cases. However, the M-F1 declines more clearly. This indicates that long windows weaken root cause separability by mixing unrelated log contexts. Therefore, we set $w = 6$ as the default value. This setting captures the short fault burst observed in real syslogs and avoids excessive contextual noise.

6) *EMA Coefficients (δ : δ_T and δ_H^u)*: The proposed FL-EMA mechanism contains two EMA coefficients with different roles. The server-side coefficient δ_T controls the sequential Transformer EMA on the VS, and the client-side coefficient δ_H^u controls the temporal smoothing of the personalized head on client u . We conduct a two-stage sensitivity analysis [71] on the two coefficients separately.

Stage 1 (Client-Side Coefficient δ_H^u): To probe δ_H^u , we first fix the server-side coefficient at $\delta_T = 1$ and sweep δ_H^u over $\{0, 0.3, 0.5, 0.7, 0.8, 0.9, 1\}$. As shown in the last subfigures of Figs. 4 and 5, the F1 curve remains nearly flat across the whole range, which indicates that binary fault detection is insensitive to the head smoothing. In contrast, the M-F1 curve rises first and then declines, and it reaches the peak at $\delta_H^u = 0.8$ on both datasets. Meanwhile, the boundary point $\delta_H^u = 0$ yields a relatively high F1 in binary fault detection but a much lower M-F1 for RCA. The reason is that, at this boundary point, the local head performs no temporal smoothing and directly adopts the parameters trained in the current round. Since RCA is the core diagnostic target, $\delta_H^u = 0.8$ provides a better balance between fault detection and RCA.

Stage 2 (Server-Side Coefficient δ_T): To probe δ_T , we then fix $\delta_H^u = 0.8$ and sweep δ_T over the same range, as shown in Figs. 4 and 5. Both the F1 and the M-F1 curves exhibit a clear single-peak shape, and they reach the maximum at $\delta_T = 0.8$. When $\delta_T = 0$, the server-side EMA discards the accumulated state and retains only the current client parameters, so the Transformer backbone loses cross-client smoothing. When $\delta_T = 1$, the server-side EMA degenerates into plain FedAvg according to Corollary 1, so the model loses server-side personalization and the M-F1 decreases again. Therefore, the value $\delta_T = 0.8$ achieves the best balance between cross-client smoothing and personalization.

In summary, both coefficients are set to 0.8. This empirical configuration is fully consistent with the convergence analysis in Corollary 2, where a moderately large coefficient suppresses the EMA weight discrepancy term (IV) without enlarging the dominant error.

V. DOCKER DEPLOYMENT AND CASE STUDY

We have deployed *FSLog* in a containerized MaaS environment to verify its performance in fault diagnosis and privacy

protection. Compared with traditional centralized approaches, *FSLog* enables collaborative training without exposing raw syslogs, while maintaining high diagnostic performance.

A. Deployment Workflow of Docker

The workflow of *FSLog* in the production environment consists of four steps.

Step 1 (Deploy Docker): The deployment instantiates five CS Docker containers and one VS Docker container orchestrated by Docker Swarm on a single host.

Step 2 (Collection and Grouping): Each CS collects syslogs from its managed IoT nodes through the gateway. The logs are preprocessed using Drain for template extraction and grouped by component within sliding time windows to form homogeneous log sequences.

Step 3 (Federated Split Training and Diagnosis): The preprocessed log sequences are fed into the local E-layer of SL-Bert on each CS, and the protected intermediate representations are uploaded to the VS through the overlay network. The VS executes forward FedAvg aggregation and backward EMA personalization on the T-layer backbone. After training, each CS assembles its personalized local model for online fault detection and RCA.

Step 4 (Report and Notification): After *FSLog* diagnoses a fault, the system generates a fault report containing the identified root cause category and suggested mitigation actions based on historical cases. The report is then sent to O&M staff via email.

B. Case Study

This case study evaluates the Docker-based MaaS deployment of *FSLog* from two perspectives: fault diagnosis and privacy protection. We first examine whether *FSLog* can efficiently detect and diagnose injected CMCC fault cases in a containerized CS-VS topology, and then evaluate whether its privacy-preserving split representations can resist attribute inference attacks without degrading diagnostic utility.

1) *Fault Diagnosis*: To verify the practical deployability of *FSLog* in a MaaS environment, we deploy it on the CMCC OpenStack-based VSwitch system using a Docker-Swarm-based containerized topology. The CSs host personalized local models and the VS maintains the aggregated Transformer backbone. For fault-diagnosis validation, we inject 93 fault cases spanning six root cause categories into the deployment environment. During the two-week monitoring period, the VSwitch node `cmcc-compute-17` generated a sequence of abnormal OpenStack VSwitch logs containing events such as “`ovs-vswitchd|ERR| bond1: interface eth1 link state down`,” which belongs to the operational port down category. During this fault case, the operator would need to examine approximately 188 432 log entries containing these keywords to locate the failure-related logs. Traditional manual diagnosis requires operators to search through a large volume of raw logs by specifying keywords such as “Error,” “Fail,” or “Exception,” which typically requires about 30 min per case. In contrast, *FSLog* eliminates this manual effort by automatically computing the fault representation vector and classifying it into the corresponding

root cause category. To assess the algorithmic efficiency of our framework, we further compare the diagnosis time of *FSLog* against NeuralLog, the strongest automated baseline. For fairness, we deploy NeuralLog under the same containerized topology, with identical input logs, hardware, and evaluation protocol. NeuralLog processes the entire Transformer backbone locally on the resource-constrained client nodes. Due to this deployment manner, it suffers from a notable computational bottleneck and requires, on average, 14.5 min to complete the detection and analysis of a complex fault case. In contrast, *FSLog* leverages the SL-Bert architecture to offload the compute-intensive task to the VS, and thus completes the same diagnosis pipeline within 5.2 min per case on average. Relative to the automated baseline, *FSLog* reduces the diagnosis time by 64.1%. This comparison confirms the algorithmic effectiveness of our computation-offloading strategy, and it effectively prevents misdiagnosis and unnecessary system restarts.

2) *Privacy Protection*: In the same MaaS deployment, the CE typically refuses to share raw syslogs with the DV because log content may contain sensitive information, such as server IP addresses and region-level deployment information. Therefore, *FSLog* only uploads protected intermediate representations from the CS to the VS, avoiding direct exposure of raw syslogs. To evaluate deployment-time privacy protection on CMCC, we simulate two attribute inference attacks: exposing log regions and inferring server IP addresses. As reported in Table V, without any privacy chain at the default boundary split ($k = 0$), the ASRs reach 45% for CE_1 (Expose area) and 40% for CE_Ω (Infer server IP). Under the full *FSLog* privacy chain at the same split point, the corresponding ASRs are reduced to 20% and 25%, respectively. Thus, *FSLog* decreases the ASRs by 25 and 15 percentage points, corresponding to relative reductions of 55.6% and 37.5%. Meanwhile, the downstream diagnosis performance remains high, with an M-F1 of 0.960 and only a 1.5% relative utility loss compared with the unprotected setting, whose M-F1 is 0.975. These results demonstrate that *FSLog* supports practical Docker-based fault diagnosis while substantially reducing sensitive syslog exposure in MaaS deployments.

VI. CONCLUSION AND FUTURE PROSPECTS

This study introduces *FSLog*, a fault diagnosis framework based on FSL, capable of efficient fault detection and precise RCA in large-scale IoT deployments integrated with cloud infrastructure. The FL-EMA strategy enables personalized fine-tuning and local model updates, enhancing model generalization, which is substantiated by our nonconvex theoretical convergence analysis. SL-Bert effectively preserves client data privacy. Experiments in a Docker-based environment demonstrate that *FSLog* outperforms baseline methods in predictive accuracy and load balancing. In future work, we will further deploy the client-side components on physical resource-constrained IoT hardware and report measured on-device latency and memory usage, and we will extend the evaluation to syslogs collected directly from typical IoT end devices.

APPENDIX PROOF OF LEMMA 1

Proof: We expand (10) by following the server-side EMA recursion on the VS. We then substitute the FedAvg aggregation in (9) into the closed-form expansion and collect all terms according to the ordered client index. In this proof, r denotes the current communication round, and $r + 1$ denotes the updated state after FedAvg and EMA. Let u_j be the j th client in the sequential EMA order, where $j = 1, \dots, N$. Let s_j be the server-side EMA step after processing client u_j . The parameter $W_{T^r}^{u_j*}$ denotes the locally optimized Transformer parameter of client u_j in round r . The coefficient $p_{u_j} = |D_{u_j}|/|D|$ is the data proportion of client u_j , where $|D| = \sum_{j=1}^N |D_{u_j}|$. The scalar $\delta_T \in [0, 1]$ is the server-side EMA coefficient. From (9), the initial EMA state is the FedAvg result $W_{T^{r+1}}^{\text{ema}(s_0)} = W_{T^{r+1}}^{\text{agg}}$. Meanwhile, from (8), the FedAvg aggregation is written as $W_{T^{r+1}}^{\text{agg}} = \sum_{j=1}^N p_{u_j} W_{T^r}^{u_j*}$. Substituting this aggregation result into the first EMA update in (10) gives the following:

$$\begin{aligned} W_{T^{r+1}}^{\text{ema}(s_1)} &= \delta_T W_{T^{r+1}}^{\text{ema}(s_0)} + (1 - \delta_T) W_{T^r}^{u_1*} \\ &= \delta_T W_{T^{r+1}}^{\text{agg}} + (1 - \delta_T) W_{T^r}^{u_1*}. \end{aligned}$$

Applying (10) once more gives the second EMA state

$$\begin{aligned} W_{T^{r+1}}^{\text{ema}(s_2)} &= \delta_T W_{T^{r+1}}^{\text{ema}(s_1)} + (1 - \delta_T) W_{T^r}^{u_2*} \\ &= \delta_T^2 W_{T^{r+1}}^{\text{agg}} + (1 - \delta_T) \delta_T W_{T^r}^{u_1*} + (1 - \delta_T) W_{T^r}^{u_2*}. \end{aligned}$$

The above two steps show that the initial FedAvg state is multiplied by one additional factor δ_T at each EMA step. In contrast, the update from client u_j is multiplied by $(1 - \delta_T) \delta_T^{N-j}$ after all N clients have been processed. We now prove this pattern by induction. Assume that after step s_j , the EMA state has the following form:

$$W_{T^{r+1}}^{\text{ema}(s_j)} = \delta_T^j W_{T^{r+1}}^{\text{agg}} + (1 - \delta_T) \sum_{k=1}^j \delta_T^{j-k} W_{T^r}^{u_k*}.$$

Substituting this expression into (10) gives

$$\begin{aligned} W_{T^{r+1}}^{\text{ema}(s_{j+1})} &= \delta_T W_{T^{r+1}}^{\text{ema}(s_j)} + (1 - \delta_T) W_{T^r}^{u_{j+1}*} \\ &= \delta_T^{j+1} W_{T^{r+1}}^{\text{agg}} + (1 - \delta_T) \sum_{k=1}^j \delta_T^{j+1-k} W_{T^r}^{u_k*} \\ &\quad + (1 - \delta_T) W_{T^r}^{u_{j+1}*} \\ &= \delta_T^{j+1} W_{T^{r+1}}^{\text{agg}} + (1 - \delta_T) \sum_{k=1}^{j+1} \delta_T^{j+1-k} W_{T^r}^{u_k*}. \end{aligned}$$

Therefore, the induction holds. After all N clients have been processed, we obtain closed-form expression in (20)

$$W_{T^{r+1}}^{\text{ema}(s_N)} = \delta_T^N W_{T^{r+1}}^{\text{agg}} + (1 - \delta_T) \sum_{j=1}^N \delta_T^{N-j} W_{T^r}^{u_j*}.$$

From (8), we have $W_{T^{r+1}}^{\text{agg}} = \sum_{j=1}^N p_{u_j} W_{T^r}^{u_j*}$. We substitute this result into (20) and collect terms by client index, which yields

$$W_{T^{r+1}}^{\text{ema}(s_N)} = \delta_T^N \sum_{j=1}^N p_{u_j} W_{T^r}^{u_j*} + (1 - \delta_T) \sum_{j=1}^N \delta_T^{N-j} W_{T^r}^{u_j*}$$

$$= \sum_{j=1}^N \left[\delta_T^N p_{u_j} + (1 - \delta_T) \delta_T^{N-j} \right] W_{T'}^{u_j*}.$$

Let $\gamma_{u_j} = \delta_T^N p_{u_j} + (1 - \delta_T) \delta_T^{N-j}$. Then, the final server-side EMA result can be written as

$$W_{T'+1}^{\text{ema}} = W_{T'+1}^{\text{ema}(s_N)} = \sum_{j=1}^N \gamma_{u_j} W_{T'}^{u_j*}.$$

This expression is exactly the effective aggregation form in (18) and (19). It remains to verify that the effective weights are normalized. For $0 < \delta_T < 1$, we obtain (21) as follows:

$$\begin{aligned} \sum_{j=1}^N \gamma_{u_j} &= \delta_T^N \sum_{j=1}^N p_{u_j} + (1 - \delta_T) \sum_{j=1}^N \delta_T^{N-j} \\ &= \delta_T^N + (1 - \delta_T) \sum_{\ell=0}^{N-1} \delta_T^\ell \\ &= \delta_T^N + (1 - \delta_T) \frac{1 - \delta_T^N}{1 - \delta_T} = 1. \end{aligned}$$

The boundary cases are also well defined. When $\delta_T = 1$, (10) keeps the EMA state equal to the FedAvg result. Hence, $W_{T'+1}^{\text{ema}} = W_{T'+1}^{\text{agg}}$ and $\gamma_{u_j} = p_{u_j}$. When $\delta_T = 0$, (10) replaces the EMA state with the latest processed client parameter at each step. Hence, the final weight is assigned to u_N , and the weights still sum to one. Therefore, the FL-EMA update is a normalized weighted aggregation of locally optimized client Transformer parameters. This completes the proof. ■

DATA AVAILABILITY

The relevant experimental data and code are publicly available at <https://github.com/IoT1220/FSLog>

REFERENCES

- [1] M. Ma et al., "Jump-starting multivariate time series anomaly detection for online service systems," in *Proc. USENIX ATC*, 2021, pp. 413–426.
- [2] INFRASPEAK. (2022). *Maintenance-as-a-Service: What is MaaS?*. [Online]. Available: <https://blog.infraspeak.com/maintenance-as-a-service/>
- [3] L. Harding, F. Wernli, and F. Suter, "Sequence-RTG: Efficient and production-ready pattern mining in system log messages," in *Proc. IEEE Int. Conf. Cluster Comput. (CLUSTER)*, Sep. 2021, pp. 623–631.
- [4] D. C. Nguyen, M. Ding, P. N. Pathirana, A. Seneviratne, J. Li, and H. V. Poor, "Federated learning for Internet of Things: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 3, pp. 1622–1658, 3rd Quart., 2021.
- [5] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. Y. Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, vol. 54, A. Singh and J. Zhu. Eds., Fort Lauderdale, FL, USA, 2017, pp. 1273–1282.
- [6] Z. Tang, Y. Zhang, S. Shi, X. He, B. Han, and X. Chu, "Virtual homogeneity learning: Defending against data heterogeneity in federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2022, pp. 21111–21132.
- [7] D. Nickel, F. P. Lin, S. Hosseinalipour, N. Michelusi, and C. G. Brinton, "Resource-efficient and delay-aware federated learning design under edge heterogeneity," in *Proc. IEEE Int. Conf. Commun. Workshops (ICC Workshops)*, May 2022, pp. 43–48.
- [8] J. Wang et al., "A field guide to federated optimization," 2021, *arXiv:2107.06917*.
- [9] Z. Zhang, N. He, W. Huo, X. Xu, C. Sun, and J. Li, "Privacy preserving federated learning for proton exchange membrane fuel cell," *Renew. Sustain. Energy Rev.*, vol. 212, Apr. 2025, Art. no. 115407.
- [10] T. Zhang, C. He, T. Ma, L. Gao, M. Ma, and S. Avestimehr, "Federated learning for Internet of Things," in *Proc. ACM Conf. Embedded Netw. Sensor Syst. (SenSys)*, 2021, pp. 413–419.
- [11] B. Li, S. Ma, R. Deng, K. R. Choo, and J. Yang, "Federated anomaly detection on system logs for the Internet of Things: A customizable and communication-efficient approach," *IEEE Trans. Netw. Service Manage.*, vol. 19, no. 2, pp. 1705–1716, Jun. 2022.
- [12] T.-H. Shin and S.-H. Kim, "Utility analysis about log data anomaly detection based on federated learning," *Appl. Sci.*, vol. 13, no. 7, p. 4495, Apr. 2023.
- [13] K. M. N. Shaik, "Harnessing artificial intelligence to strengthen intrusion detection in modern network system," *Int. J. Scientific Res. Manage. (IJSRM)*, vol. 13, no. 7, pp. 2424–2433, Jul. 2025.
- [14] A. Sleem, "Intelligent and secure detection of cyber-attacks in industrial Internet of Things: A federated learning framework," *J. Intell. Syst. Internet Things*, vol. 7, no. 1, pp. 51–61, 2022.
- [15] W. Huang, T. Li, D. Wang, S. Du, J. Zhang, and T. Huang, "Fairness and accuracy in horizontal federated learning," *Inf. Sci.*, vol. 589, pp. 170–185, Apr. 2022.
- [16] G. N. N. Barbosa and D. M. F. Mattos, "A practical evaluation of a federated learning application in IoT device and cloud environment," *J. Brazilian Comput. Soc.*, vol. 31, no. 1, pp. 1–10, Jan. 2025.
- [17] M. Ragab et al., "Advanced artificial intelligence with federated learning framework for privacy-preserving cyberthreat detection in IoT-assisted sustainable smart cities," *Sci. Rep.*, vol. 15, no. 1, p. 4470, Feb. 2025.
- [18] A. Vyas, P.-C. Lin, R.-H. Hwang, and M. Tripathi, "Privacy-preserving federated learning for intrusion detection in IoT environments: A survey," *IEEE Access*, vol. 12, pp. 127018–127050, 2024.
- [19] A. Bataineh, H. Alqudah, H. B. Abdoh, and F. Fataftah, "Big data-enabled federated learning for secure and collaborative industrial IoT in Industry 4.0," in *Proc. 1st Int. Conf. Comput. Intell. Approaches Appl. (ICCIAA)*, Apr. 2025, pp. 1–8.
- [20] M. Xu et al., "Multiagent federated reinforcement learning for secure incentive mechanism in intelligent cyber-physical systems," *IEEE Internet Things J.*, vol. 9, no. 22, pp. 22095–22108, Nov. 2022.
- [21] S. Mittal and J. S. Vetter, "A survey of CPU-GPU heterogeneous computing techniques," *ACM Comput. Surveys*, vol. 47, no. 4, pp. 1–35, Jul. 2015.
- [22] C. T. Dinh, N. H. Tran, and T. D. Nguyen, "Personalized federated learning with Moreau envelopes," in *Proc. Adv. Neural Inf. Process. Syst.*, Dec. 2020, pp. 21394–21405.
- [23] S. Abuadba et al., "Can we use split learning on 1D CNN models for privacy preserving training?" in *Proc. 15th ACM Asia Conf. Comput. Commun. Secur.*, Oct. 2020, pp. 305–318.
- [24] C. Thapa, P. C. M. Arachchige, S. Camtepe, and L. Sun, "SplitFed: When federated learning meets split learning," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 8485–8493.
- [25] Y. Tian, Y. Wan, L. Lyu, D. Yao, H. Jin, and L. Sun, "FedBERT: When federated learning meets pre-training," *ACM Trans. Intell. Syst. Technol.*, vol. 13, no. 4, pp. 1–26, Aug. 2022.
- [26] S. Kariyappa and M. K. Qureshi, "Exploit: Extracting private labels in split learning," in *Proc. IEEE Conf. Secure Trustworthy Mach. Learn. (SaTML)*, Feb. 2023, pp. 165–175.
- [27] C. Liu, T. J. Chua, and J. Zhao, "Time minimization in hierarchical federated learning," in *Proc. IEEE/ACM 7th Symp. Edge Comput. (SEC)*, Seattle, WA, USA, Dec. 2022, pp. 96–106.
- [28] I. Ceballos et al., "SplitNN-driven vertical partitioning," 2020, *arXiv:2008.04137*.
- [29] Z. An, T. Zhang, Y. Wei, and M. Peng, "Low latency multi-dimensional offloading in intelligent transportation system," in *Proc. IEEE 21st Int. Conf. Commun. Technol. (ICCT)*, Oct. 2021, pp. 1072–1077.
- [30] Y. Gao et al., "End-to-end evaluation of federated learning and split learning for Internet of Things," 2020, *arXiv:2003.13376*.
- [31] P. Notaro, J. Cardoso, and M. Gerndt, "A survey of AIOPs methods for failure management," *ACM Trans. Intell. Syst. Technol.*, vol. 12, no. 6, pp. 1–45, Dec. 2021.
- [32] A. Venkata Mahesh, "Design of an iterative method for dynamic resource management in 5G networks with IoT integration operations," *J. Electr. Syst.*, vol. 20, no. 5s, pp. 2551–2569, Apr. 2024.
- [33] B. Prasad, R. Jopate, P. Savita, A. Basi Reddy, B. P. Shankar, and M. S. Arunkumar, "Enhancing IoT-edge computation with data forward-based decentralized deep neural networks," in *Proc. 7th Int. Conf. Image Inf. Process. (ICIIP)*, Nov. 2023, pp. 417–423.
- [34] J. Devlin, M. W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NaacL-HLT*, vol. 1, 2019, p. 2.
- [35] W. Meng et al., "LogAnomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs," in *Proc. 28th Int. Joint Conf. Artif. Intell.*, Aug. 2019, pp. 4739–4745.

- [36] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 1285–1298.
- [37] X. Zhang et al., "Robust log-based anomaly detection on unstable log data," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Aug. 2019, pp. 807–817.
- [38] S. Huang et al., "HitAnomaly: Hierarchical transformers for anomaly detection in system log," *IEEE Trans. Netw. Service Manage.*, vol. 17, no. 4, pp. 2064–2076, Dec. 2020.
- [39] T. Wittkopp et al., "A2Log: Attentive augmented log anomaly detection," 2021, *arXiv:2109.09537*.
- [40] S. Nedelkoski, J. Bogatinovski, A. Acker, J. Cardoso, and O. Kao, "Self-attentive classification-based anomaly detection in unstructured logs," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, Nov. 2020, pp. 1196–1201.
- [41] V.-H. Le and H. Zhang, "Log-based anomaly detection without log parsing," in *Proc. 36th IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Nov. 2021, pp. 492–504.
- [42] H. Guo, S. Yuan, and X. Wu, "LogBERT: Log anomaly detection via BERT," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2021, pp. 1–8.
- [43] T. Wittkopp and A. Acker, "Decentralized federated learning preserves model and data privacy," in *Proc. Int. Conf. Service-Oriented Comput.*, 2020, pp. 176–187.
- [44] Q. Lin, H. Zhang, J.-G. Lou, Y. Zhang, and X. Chen, "Log clustering based problem identification for online service systems," in *Proc. IEEE/ACM 38th Int. Conf. Softw. Eng. Companion (ICSE-C)*, May 2016, pp. 102–111.
- [45] Y. Yuan, W. Shi, B. Liang, and B. Qin, "An approach to cloud execution failure diagnosis based on exception logs in OpenStack," in *Proc. IEEE 12th Int. Conf. Cloud Comput. (CLOUD)*, Jul. 2019, pp. 124–131.
- [46] L. Liao, K. Zhu, J. Luo, and J. Cai, "LogBASA: Log anomaly detection based on system behavior analysis and global semantic awareness," *Int. J. Intell. Syst.*, vol. 2023, no. 1, Jan. 2023, Art. no. 3777826.
- [47] Y. Sui et al., "LogKG: Log failure diagnosis through knowledge graph," *IEEE Trans. Services Comput.*, vol. 16, no. 5, pp. 3493–3507, Sep. 2023.
- [48] A. Vervaet, "MoniLog: An automated log-based anomaly detection system for cloud computing infrastructures," in *Proc. IEEE 37th Int. Conf. Data Eng. (ICDE)*, Apr. 2021, pp. 2739–2743.
- [49] X. Li, P. Chen, L. Jing, Z. He, and G. Yu, "SwissLog: Robust and unified deep learning based log anomaly detection for diverse faults," in *Proc. IEEE 31st Int. Symp. Softw. Rel. Eng. (ISSRE)*, Oct. 2020, pp. 92–103.
- [50] Y. Yang and X. Peng, "BERT-based network for intrusion detection system," *EURASIP J. Inf. Secur.*, vol. 2025, no. 1, p. 11, Mar. 2025.
- [51] D. Menegatti, E. De Santis, S. Felli, and A. Giuseppi, "AdaLightLog: Enhancing application logs anomaly detection via adaptive federating learning," in *Proc. Int. Conf. Crit. Inf. Infrastructures Secur.*, 2024, pp. 289–305.
- [52] H.-T. Nguyen, L.-V. Nguyen, V.-H. Le, H. Zhang, and M.-T. Le, "Efficient log-based anomaly detection with knowledge distillation," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, Jul. 2024, pp. 578–589.
- [53] D. A. E. Acar, Y. Zhao, R. Matas, M. Mattina, P. N. Whatmough, and V. Saligrama, "Federated learning based on dynamic regularization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2021. [Online]. Available: <https://openreview.net/forum?id=B7v4QMR6Z9w>
- [54] J. Wang, Q. Liu, H. Liang, G. Joshi, and V. H. Poor, "Tackling the objective inconsistency problem in heterogeneous federated optimization," in *Proc. NIPS*, 2020, pp. 7611–7623.
- [55] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst.*, vol. 2, 2020, pp. 429–450.
- [56] Y. Xia, Y. Zhao, J. Wan, and C. Jiang, "FLUXLog: A federated mixture-of-experts framework for unified log anomaly detection," *IEEE Trans. Parallel Distrib. Syst.*, vol. 37, no. 2, pp. 395–409, Feb. 2026.
- [57] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 5132–5143.
- [58] E. Samikwa, A. Di Maio, and T. Braun, "DFL: Dynamic federated split learning in heterogeneous IoT," *IEEE Trans. Mach. Learn. Commun. Netw.*, vol. 2, pp. 733–752, 2024.
- [59] J. Shin, J. Ahn, H. Kang, and J. Kang, "FedSplitX: Federated split learning for computationally-constrained heterogeneous clients," 2023, *arXiv:2310.14579*.
- [60] W. Guan, J. Cao, H. Zhao, Y. Gu, and S. Qian, "FeadSeq: A personalized federated anomaly detection framework for discrete event sequences," *ACM Trans. Knowl. Discovery from Data*, vol. 19, no. 6, pp. 1–26, Jul. 2025.
- [61] Y. Liao, J. Keung, Z. Mao, J. Zhang, and J. Li, "FedLAD: A modular and adaptive testbed for federated log anomaly detection," in *Proc. IEEE Int. Conf. Autonomic Comput. Self-Organizing Syst. Companion (ACSOS-C)*, Sep. 2025, pp. 227–232.
- [62] J. Ramos, "Using TF-IDF to determine word relevance in document queries," in *Proc. 1st Instructional Conf. Mach. Learn.*, vol. 242, no. 1, 2003, pp. 29–48.
- [63] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 26, 2013, pp. 3111–3119.
- [64] J. Li, M. S. Othman, H. Chen, and L. M. Yusuf, "Optimizing IoT intrusion detection system: Feature selection versus feature extraction in machine learning," *J. Big Data*, vol. 11, no. 1, p. 36, Feb. 2024.
- [65] I. Hasan, A. All Ahhad, S. Md Rakibul Islam, M. A. Mim, and M. Habibur Rahman, "Enhancing intrusion detection capabilities in IoT technologies through machine learning," in *Proc. IEEE Int. Conf. Comput., Appl. Syst. (COMPAS)*, Sep. 2024, pp. 1–6.
- [66] X. Liu, J. Yu, J. Wang, and Y. Gao, "Resource allocation with edge computing in IoT networks via machine learning," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 3415–3426, Apr. 2020.
- [67] N. M. Eldakhly, "Optimized intrusion detection with deep learning classification models," *Neural Comput. Appl.*, vol. 37, no. 25, pp. 21091–21119, Sep. 2025.
- [68] X. Li, K. Huang, W. Yang, S. Wang, and Z. Zhang, "On the convergence of FedAvg on non-IID data," 2019, *arXiv:1907.02189*.
- [69] T. Rosado and J. Bernardino, "An overview of OpensTack architecture," in *Proc. 18th Int. Database Eng. Appl. Symp.*, 2014, pp. 366–367.
- [70] M. Nasr, R. Shokri, and A. Houmansadr, "Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2019, pp. 739–753.
- [71] B. Li, S. Zhang, and Q. Han, "Federated learning with joint server-client momentum," *Sci. Rep.*, vol. 15, no. 1, p. 15626, May 2025.



Ting Xu (Member, IEEE) received the M.E. degree in software engineering from Central South University, Changsha, China, in 2020, and the Ph.D. degree in software engineering from Nankai University, Tianjin, China, in 2026.

From 2025 to 2026, she was a Visiting Ph.D. student with the University of Waterloo, Waterloo, Canada. Her research interests include federated learning, deep learning, anomaly detection, failure diagnosis, and large language models.



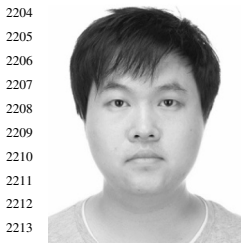
Shenglin Zhang (Member, IEEE) received the B.S. degree in network engineering from the School of Computer Science and Technology, Xidian University, Xi'an, China, in 2012, and the Ph.D. degree in computer science from Tsinghua University, Beijing, China, in 2017.

He is currently an Associate Professor with the College of Software, Nankai University, Tianjin, China. His current research interests include failure detection, diagnosis, and prediction for service management.



Zeyu Che received the B.S. degree in software engineering from Nankai University, Tianjin, China, in 2023, where he is currently pursuing the M.S. degree with the College of Software.

His research interests include deep learning and anomaly detection.



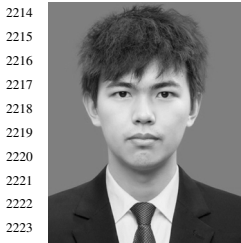
Yongqian Sun (Member, IEEE) received the B.S. degree in statistical specialty from Northwestern Polytechnical University, Xi'an, China, in 2012, and the Ph.D. degree in computer science from Tsinghua University, Beijing, China, in 2018.

He is currently an Assistant Professor with the College of Software, Nankai University, Tianjin, China. His research interests include anomaly detection and root cause localization in service management.



Zhimin Chen is currently pursuing the B.E. degree in software engineering with the College of Software, Nankai University, Tianjin, China.

His current interests include AIOps and large language model agents.



Wenwei Gu received the B.S. degree in electronic information engineering from the Huazhong University of Science and Technology, Wuhan, China, in 2021, and the Ph.D. degree in computer science and engineering from The Chinese University of Hong Kong, Hong Kong, China, in 2025.

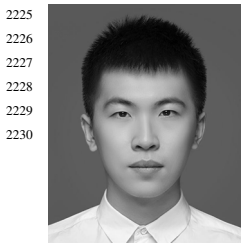
He is currently a Lecturer with the College of Software, Nankai University, Tianjin, China. His current research interests include multiagent-driven intelligent operations, reliability of AI infrastructure, and reliability of large language model training.



Weiye Shang (Senior Member, IEEE) received the B.Eng. degree from Harbin Institute of Technology, Harbin, China, in 2008, and the M.Sc. and Ph.D. degrees in computer science from Queen's University, Kingston, ON, Canada, in 2010 and 2014, respectively.

He is currently a Professor with the Department of Electrical and Computer Engineering, University of Waterloo, Waterloo, ON, Canada. His research interests include software engineering, with a particular focus on ultralarge-scale systems, software

log mining, empirical software engineering, mining software repositories, and performance engineering.



Yicheng Sui received the B.S. degree in software engineering from Nankai University, Tianjin, China, in 2020, where he is currently pursuing the Ph.D. degree with the College of Software.

His current research interests include machine learning and deep learning.



Kundi Yao received the M.A.Sc. degree in software engineering from Concordia University, Montreal, QC, Canada, in 2018, and the Ph.D. degree in software engineering from Queen's University, Kingston, ON, Canada, in 2024.

He is currently an Assistant Professor with Ontario Tech University, Oshawa, ON, Canada. His research interests include AIOps, log analytics, software performance, and big data-driven software engineering, with the goal of improving the quality, observability, and maintainability of large-scale

software systems. For more information, see <https://kdyao.github.io/>



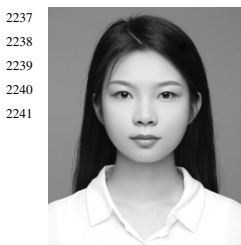
Sibao Xia received the B.S. degree in software engineering from the College of Software, Nankai University, Tianjin, China, in 2022, where he is currently pursuing the Ph.D. degree.

His research interests include failure detection, diagnosis, and prediction.



Jiayi Ma received the B.Math. degree in statistics from the University of Waterloo, Waterloo, ON, Canada, in 2017, where she is currently pursuing the M.A.Sc. degree in electrical and computer engineering.

Her research interests include behavioral log analysis, user intent modeling, web analytics, and causal inference.



Zijiang Que is currently pursuing the B.S. degree with the College of Software, Nankai University, Tianjin, China.

Her research interests include deep learning and anomaly detection.



Dan Pei (Senior Member, IEEE) received the B.E. and M.S. degrees in computer science from the Department of Computer Science and Technology, Tsinghua University, Beijing, China, in 1997 and 2000, respectively, and the Ph.D. degree in computer science from the Department of Computer Science, University of California, Los Angeles (UCLA), Los Angeles, CA, USA, in 2005.

He is currently an Associate Professor with the Department of Computer Science and Technology, Tsinghua University. His research interests include

network and service management in general.

Dr. Pei is a Senior Member of ACM.